

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)



МЕТОДИЧЕСКИЕ УКАЗАНИЯ

**по выполнению лабораторных работ по дисциплине
«Программная инженерия»:
часть II – технология разработки программных систем**

для бакалавров направления подготовки 02.03.03 «Математическое обеспечение и администрирование информационных систем» (профиль «Технологии разработки программных систем») и 09.03.04 «Программная инженерия» (профиль «Разработка программного обеспечения и приложений искусственного интеллекта»)

Краснодар,
2026

Методические указания к выполнению лабораторных работ по дисциплине «Программная инженерия»: часть II – технология разработки программных систем, для бакалавров направления подготовки 02.03.03 «Математическое обеспечение и администрирование информационных систем» (профиль «Технологии разработки программных систем») и 09.03.04 «Программная инженерия» (профиль «Разработка программного обеспечения и приложений искусственного интеллекта») / Составители: доц. каф. ИТ Полетайкин А.Н., доц. каф. ИТ Добровольская Н.Ю. – Краснодар: КубГУ, 2026. – 60 с.

Методические указания содержат теоретические положения, задания и методические указания к выполнению лабораторных работ по дисциплине «Программная инженерия» во втором семестре изучения дисциплины. Предметом изучения является технология разработки программных систем.

Составители: доц. каф. ИТ, к.т.н., доц. Полетайкин А.Н.

доц. каф. ИТ, к.пед.н., доц. Добровольская Н.Ю.

СОДЕРЖАНИЕ

1	Общие положения	5
1.1	Организация занятий по курсу	5
1.2	Требования к содержанию отчетов о выполнении лабораторных работ.....	8
1.3	Требования к оформлению отчетов.....	8
2	Методические указания	11
	Лабораторная работа 1	11
	Задание	11
	Теоретические положения.....	11
	1.1. Структура эскизного проекта ПС	11
	1.2. Системный подход к описанию объекта и процесса компьютеризации	12
	1.3. Описание информационных потоков	14
	1.4. Постановка задачи на создание ПС	15
	Рекомендуемая литература	17
	Контрольные вопросы	17
	Лабораторная работа 2	18
	Задание	18
	Требования к содержанию отчета.....	18
	Рекомендуемая литература	19
	Контрольные вопросы	19
	Лабораторная работа 3	21
	Задание	21
	Требования к содержанию отчета.....	21
	Рекомендуемая литература	22
	Контрольные вопросы	23
	Лабораторная работа 4	24
	Задание	24
	Теоретические положения.....	25
	4.1. CASE-средство ERwin для моделирования баз данных.....	25
	4.2. Разработка логической модели данных	26
	4.3. Разработка физической модели данных.....	28
	Рекомендуемая литература	32
	Контрольные вопросы	32
	Лабораторная работа 5	33
	Задание	33
	Теоретические положения.....	35
	5.1. Классификация программного обеспечения ПС	35
	5.2. Структура баг-репорта	36
	Требования к содержанию отчета.....	37
	Рекомендуемая литература	38
	Контрольные вопросы	38

Лабораторная работа 6	39
Задание	39
Теоретические положения.....	39
6.1. Виды тестирования ПО	39
6.2. Сценарное тестирование	41
6.3. Интеграционное тестирование	41
6.4. Системное тестирование.....	42
Рекомендуемая литература	42
Контрольные вопросы	43
Лабораторная работа 7	44
Задание	44
Теоретические положения.....	44
7.1. Документирование ПО.....	44
7.2. Руководство пользователя	45
Рекомендуемая литература	46
Контрольные вопросы	46
Лабораторная работа 8	47
Задание	47
Теоретические положения.....	47
8.1 Методика риск-менеджмента программного процесса.....	47
Требования к содержанию отчета.....	48
8.2. Идентификация рисков	48
8.3. Качественное оценивание рисков.....	49
8.4. Воздействие на риски.....	50
Рекомендуемая литература	51
Контрольные вопросы	51
Список рекомендуемой литературы	52
Приложение А – Варианты индивидуальных заданий	54
Приложение Б – Примерное описание объекта компьютеризации.....	57
Приложение В – Образец титульного листа.....	60

1 Общие положения

Дисциплина «Программная инженерия» рассматривает методы, способы, модели и инструментальные средства для решения задач создания качественного программного обеспечения (ПО), программных продуктов (ПП) и программных систем (ПС) в разных сферах деятельности человека с применением современных компьютерных информационных технологий.

Изучаются основные понятия программной инженерии, составляющие процесса разработки ПО и ПП, управление требованиями к ПО, управление конфигурацией и качеством ПП. Рассматриваются и применяются на практике методы, способы и инструментальные средства для анализа предметных областей, постановки задачи на разработку ПП и ПС, формулирования требований к ним, компилятивной сборки и комплексного тестирования специального ПО для потребностей цифровой экономики с учетом задач будущей профессиональной деятельности студентов.

1.1 Организация занятий по курсу

Дисциплина «Программная инженерия» читается студентам направлений подготовки 02.03.03 «Математическое обеспечение и администрирование информационных систем» (профиль «Технологии разработки программных систем») и 09.03.04 «Программная инженерия» (профиль «Разработка программного обеспечения и приложений искусственного интеллекта») на третьем курсе обучения в осеннем и весеннем семестрах. Базовыми учебными дисциплинами для изучения дисциплины «Программная инженерия» являются «Методы программирования», «Низкоуровневое программирование», «Объектно-ориентированное программирование», «Алгоритмы и анализ сложности», «Компьютерные сети», «Базы данных», «Анализ, проектирование и разработка БД», «Операционные системы».

Целью курса является формирование у студентов знаний, умений и практических навыков в области анализа и системного представления бизнес-процессов, разработки технического задания на разработку ПП и ПС для решения задач человека в разных областях, а также организации программного процесса для создания ПП и ПС специального назначения.

Предметом учебной дисциплины являются методы, подходы и инструментальные средства программной инженерии.

Задачами дисциплины является получение представления о жизненном цикле многоверсионного программного продукта, а также приобретения навыков применения знаний и умений, приобретенных на младших курсах бакалавриата, для создания и сопровождения сложных программных проектов, отвечающих требованиям цифровой экономики.

Изучаемый курс предусматривает выполнение 17 лабораторных работ (9 работ в осеннем и 8 работ в весеннем семестре). В осеннем семестре выполняется

разработка ПП для реализации обработки информации и вычислений на ЭВМ в рамках заданного бизнес-процесса ([см. приложение А](#)).

Рабочая схема выполнения лабораторных работ в осеннем семестре представлена в таблице 1.1.

Таблица 1.1. Рабочая схема выполнения лабораторных работ в осеннем семестре

№ л. р.	Тема лабораторной работы: материал для изучения и характер выполняемых работ	Объем, часов	Неделя выполнения
1.	<u>Анализ предметной области</u> : системное описание заданного бизнес-процесса, характеристика ручного режима решения задач и выделение его недостатков, обоснование усовершенствования существующей схемы решения задач	5	1–3
2.	<u>Анализ существующих подобных программных решений</u> : системное описание существующих подобных решений, выделение их достоинств и недостатков, сравнительная характеристика	3	3, 4
3.	<u>Техническое задание на создание программного продукта</u> : назначение и общая цель создания ПП, структура ПП, функциональные и нефункциональные требования к ПП, моделирование требований на языке UML	4	5, 6
4.	<u>Разработка функциональной структуры программного продукта (функционально-ориентированный подход)</u> : построение и документирование функциональной модели ПП в виде иерархии диаграмм в нотации IDEF0	3	7, 8
5.	<u>Разработка концептуальной структуры программного продукта (объектно-ориентированный подход)</u> : перечисление функций ПП, формирование сценариев работы ПП, построение диаграммы классов ПП на языке UML	4	8–10
6.	<u>Разработка функциональной структуры программного продукта (объектно-ориентированный подход)</u> : построение модели ПП на языке UML в виде комплекса диаграмм состояний, деятельности и взаимодействия	3	10, 11
7.	<u>Разработка архитектуры программного продукта</u> : проектирование компонентного состава ПП, разработка структур данных для ПП, построение диаграммы компонентов UML и диаграммы развертывания UML	4	12, 13
8.	<u>Разработка специального программного обеспечения</u> : выбор технологии программирования, разработка и отладка компонентов ПП, анализ сложности и эффективности кода, проведение рефакторинга	4	14, 15
9.	<u>Комплексное тестирование программного продукта</u> : ручное и автоматическое тестирование, проверка работоспособности ПП, модульное, регрессионное и нагрузочное тестирование, анализ производительности	4	16, 17
Всего за семестр:		34	

В весеннем семестре выполняется проектирование и разработка программной системы по индивидуальной теме, выбранной студентом самостоятельно при консультационной поддержке преподавателя, ведущего лабораторные занятия по дисциплине, а также руководителя курсовой работы. Рабочая схема выполнения лабораторных работ в весеннем семестре представлена в таблице 1.2.

Таблица 1.2. Рабочая схема выполнения лабораторных работ в весеннем семестре

№ л. р.	Тема лабораторной работы: материал для изучения и характер выполняемых работ	Объем, часов	Неделя выполнения
1.	<u>Эскизное проектирование ПС</u> : структура объекта и процесса компьютеризации, состав функциональных задач, назначение и цель создания ПС, анализ существующих подобных ПС, требования к ПС, планирование проекта	4	1, 2
2.	<u>Проектирование функциональной структуры ПС</u> : построение и документирование комплексной функциональной модели ПС в нотации IDEF0 и UML	4	3, 4
3.	<u>Формирование среды разработки ПС</u> : конфигурационное управление ПС, развертывание, настройка и конфигурирование рабочей среды для разработки ПС, создание репозитория для управления версиями ПС, идентификация проектных рисков	4	5, 6
4.	<u>Разработка информационного обеспечения ПС</u> : выбор программных средств моделирования структур данных ПС, проработка методов нормализации отношений в БД, создание БД ПС при помощи выбранной СУБД	4	7, 8
5.	<u>Разработка программного обеспечения ПС</u> : технология программирования прикладных задач, разработка интерфейсной части ПС, построение диаграмм UML физической реализации ПС, модульное и системное тестирование ПО, анализ сложности кода ПО	4	9, 10
6.	<u>Комплексное тестирование ПС</u> : интеграционное и нагрузочное тестирование, тестирование взаимодействия, пользовательское тестирование	4	11, 12
7.	<u>Документирование и развертывание ПС</u> : освоение методики документирования ПС, разработка руководства пользователя ПС, освоение технологии развертывания ПС	4	13, 14
8.	<u>Управление рисками ПС</u> : идентификация, анализ и картографирование рисков факторов на стадиях жизненного цикла ПС, выработка необходимых мероприятий для нормализации рисков факторов ПС	4	15, 16
	Всего за семестр:	32	

По каждой лабораторной работе оформляется отчет. Отчеты сдаются на проверку преподавателю, ведущему лабораторные занятия, в течение курса по мере

их выполнения и защищаются студентами в установленном порядке. Защищенные отчеты загружаются в ЭИОС.

При защите отчета студенту могут быть заданы вопросы и дополнительные задания по теме лабораторной работы, в том числе из списка контрольных вопросов к данной лабораторной работе. При неудовлетворительной оценке знаний студента по теме данного отчета студент возвращается к повторному изучению соответствующих материалов, после чего допускается к повторной защите. Неудовлетворительно выполненный отчет также возвращается на доработку. Требования к содержанию отчетов о выполнении лабораторных работ представлены в [разделе 1.2](#).

1.2 Требования к содержанию отчетов о выполнении лабораторных работ

В процессе изучения курса «Программная инженерия» должны быть подготовлены и сданы 16 отчетов о выполнении лабораторных работ (см. таблицы 1.1 и 1.2). Лабораторные работы выполняются в соответствии с заданием ([раздел 2](#)) в рамках индивидуальной темы.

Отчет предваряется титульным листом установленного образца (см. [приложение В](#)) и содержит тему лабораторной работы, цель, задание, индивидуальную тему, описание хода выполнения работы, необходимые прикладные материалы (схемы, макеты документов и т.п.), в соответствии с требованиями к содержанию, представленными в соответствующем подразделе раздела 2, и выводы по работе.

Отчет о выполнении каждой без исключения лабораторной работы предполагает включение в него иллюстраций (рисунков, схем, графиков, диаграмм и др.). Также большинство отчетов предполагает наличие таблиц.

Для корректного выполнения и успешной защиты отчетов необходимо перед выполнением каждой лабораторной работы внимательно и полностью прочитать задание и требования к содержанию отчета, и только после этого приступить к выполнению работы и оформлению отчета. После завершения работы над отчетом рекомендуется еще раз свериться с заданием (которое должно располагаться в начале отчета) и убедиться в том, что все пункты задания выполнены.

1.3 Требования к оформлению отчетов

Отчет должен быть выполнен печатным способом с использованием компьютера и принтера на одной стороне листа белой бумаги формата А4 через полтора интервала. Для создания текста работы рекомендуется использовать текстовый редактор MS Word. Шрифт на протяжении всего документа должен быть одинаковый: Times New Roman 14-го размера, за исключением оформления иллюстраций, таблиц и формул, в которых допускается использовать шрифт меньшего размера. Минимально допустимый размер шрифта – 12.

Текст отчета следует печатать, соблюдая следующие размеры полей: слева – 25-30 мм, справа – 10 мм, сверху и снизу – 20 мм.

При оформлении текста работы следует использовать абзацный отступ, который должен составлять 15-17 мм от левого поля документа

Вне зависимости от способа выполнения отчета качество напечатанного текста и оформления иллюстраций, таблиц, распечаток с ПЭВМ должно удовлетворять требованию их четкого воспроизведения.

Иллюстрации (чертежи, графики, схемы, компьютерные распечатки, диаграммы, фотоснимки) следует располагать в отчете непосредственно после текста, в котором они упоминаются впервые, или на следующей странице. Иллюстрации могут быть в компьютерном исполнении, в том числе и цветные. На все иллюстрации должны быть даны ссылки в отчете.

Иллюстрации следует нумеровать арабскими цифрами сквозной нумерацией. Если рисунок один, то он обозначается “Рисунок 1”. Слово “Рисунок” и его наименование располагают посередине строки. Допускается нумеровать иллюстрации в пределах раздела. В этом случае номер иллюстрации состоит из номера раздела и порядкового номера иллюстрации, разделенных точкой. Например, Рисунок 1.1. Иллюстрации, при необходимости, могут иметь наименование и пояснительные данные (подрисовочный текст). Слово “Рисунок” и наименование помещают после пояснительных данных и располагают следующим образом: Рисунок 1.1 – Функциональная схема.

При ссылках на иллюстрации следует писать “... в соответствии с рисунком 2” при сквозной нумерации и “... в соответствии с рисунком 1.2” при нумерации в пределах раздела.

Таблицы применяют для лучшей наглядности и удобства сравнения показателей. Название таблицы должно отражать ее содержание, быть точным, кратким. Название таблицы следует помещать над таблицей слева, без абзацного отступа в одну строку с ее номером через тире.

При переносе части таблицы название помещают только над первой частью таблицы, нижнюю горизонтальную черту, ограничивающую таблицу, не проводят.

Таблицу следует располагать в отчете непосредственно после текста, в котором она упоминается впервые, или на следующей странице. На все таблицы должны быть ссылки в отчете. При ссылке следует писать слово “таблица” с указанием ее номера.

Таблицу с большим количеством строк допускается переносить на другой лист (страницу). При переносе части таблицы на другой лист (страницу) слово “Таблица” и номер ее указывают один раз справа над первой частью таблицы, над другими частями пишут слово “Продолжение” и указывают номер таблицы, например: “Продолжение таблицы 1”. При переносе таблицы на другой лист (страницу) заголовок помещают только над ее первой частью.

Таблицу с большим количеством граф допускается делить на части и помещать одну часть под другой в пределах одной страницы. Если строки и графы таблицы выходят за формат страницы, то в первом случае в каждой части таблицы повторяется головка, во втором случае – боковик.

Если повторяющийся в разных строках графы таблицы текст состоит из одного слова, то его после первого написания допускается заменять кавычками; если из двух и более слов, то при первом повторении его заменяют словами “То же”, а далее – кавычками. Ставить кавычки вместо повторяющихся цифр, марок, знаков, математических и химических символов не допускается. Если цифровые или иные данные в какой-либо строке таблицы не приводят, то в ней ставят прочерк.

Таблицы следует нумеровать арабскими цифрами сквозной нумерацией. Допускается нумеровать таблицы в пределах раздела. В этом случае номер таблицы состоит из номера раздела и порядкового номера таблицы, разделенных точкой.

Заголовки граф и строк таблицы следует писать с прописной буквы в единственном числе, а подзаголовки граф – со строчной буквы, если они составляют одно предложение с заголовком, или с прописной буквы, если они имеют самостоятельное значение. В конце заголовков и подзаголовков таблиц точки не ставят. Таблицы слева, справа и снизу, как правило, ограничивают линиями. Допускается применять размер шрифта в таблице меньший, чем в тексте. Горизонтальные и вертикальные линии, разграничивающие строки таблицы, допускается не проводить, если их отсутствие не затрудняет пользование таблицей. Заголовки граф, как правило, записывают параллельно строкам таблицы. При необходимости допускается перпендикулярное расположение заголовков граф. Головка таблицы должна быть отделена линией от остальной части таблицы.

Формулы и уравнения следует выделять из текста в отдельную строку. Выше и ниже каждой формулы или уравнения должно быть оставлено не менее одной свободной строки. Если уравнение не уместится в одну строку, то оно должно быть перенесено после знака равенства (=) или после знаков плюс (+), минус (-), умножения (х), деления (:), или других математических знаков, причем знак в начале следующей строки повторяют. При переносе формулы на знаке, символизирующем операцию умножения, применяют знак “Х”.

Пояснение значений символов и числовых коэффициентов следует приводить непосредственно под формулой в той же последовательности, в которой они даны в формуле.

Формулы в отчете следует нумеровать порядковой нумерацией в пределах всего отчета арабскими цифрами в круглых скобках в крайнем правом положении на строке. Одну формулу обозначают – (1). Ссылки в тексте на порядковые номера формул дают в скобках. Пример – ... в формуле (1).

2 Методические указания

Лабораторная работа 1

Тема: эскизное проектирование ПС.

Цель: освоение технологии внешнего проектирования и планирования создания ПС, изучение области инженерии требований к ПС, структуры и методики разработки эскизного проекта ПС.

Задание

1. Дать характеристику объекта компьютеризации: наименование, назначение, структура, задачи, действующие лица.
2. Выполнить системное описание процесса компьютеризации и выполнить его декомпозицию на подпроцессы (задачи).
3. Построить и описать модель процесса компьютеризации «Черный ящик». Сформировать реестры инфопотоков. Выделить недостатки процесса, определяющие необходимость его компьютеризации.
4. Выполнить системное описание существующих подобных ПС (не менее двух), которые могут быть применены к данному объекту управления. Выделить основные преимущества и недостатки представленных систем.
5. На основе выделенных в пп. 3 и 4 недостатков обосновать необходимость создания ПС. Разработать ТЗ на создание ПС: определить назначение ПС, цель создания ПС, а также требования к ПС:
 - функциональные требования (также построить диаграмму требований в нотации UML и соотнести её с моделью черного ящика);
 - требования к обеспечивающим подсистемам ПС;
 - требования к ПС в целом (нефункциональные требования)
6. Определить и спланировать ресурсы для создания ПС.

Теоретические положения

1.1. Структура эскизного проекта ПС

Проектирование ПС – это процесс создания описания, необходимого для построения в заданных условиях еще не существующей ПС, а также преобразования исходного описания в окончательное на основе выполнения работ конструкторского, исследовательского, расчетного характера.

Проект ПС – это детально описанный прообраз будущей ПС. Проект ПС бывает эскизный, технический, рабочий.

Эскизный проект отражает предварительные решения по созданию ПС, является исходным материалом для дальнейшего технического проектирования и имеет следующую структуру:

1. Организационная структура объекта компьютеризации (организационного или технического объекта).

2. Системное описание бизнес-процессов, составляющих процесс компьютеризации.
3. Краткое схематичное описание задач и операций бизнес-процесса.
4. Описание документов бизнес-процесса.
5. Анализ подобных существующих решений по компьютеризации рассмотренных бизнес-процессов.
6. Перечисление и формализация функциональных требований к ПС.
7. Основные требования и приоритеты компьютеризации: требования к обеспечивающей части ПС, приоритетные нефункциональные требования.
8. Оценивание возможности и целесообразности компьютеризации.
9. Оценивание необходимых для обеспечения проекта ресурсов заказчика.
10. Предложения по созданию ПС с оценкой примерных сроков и стоимости.

1.2. Системный подход к описанию объекта и процесса компьютеризации

Компьютеризация – это направленный процесс системной интеграции компьютерных средств, информационных и коммуникационных технологий с целью выработки эффективного управления в области применения и получения новых общесистемных свойств этой области, позволяющих более эффективно организовать процесс компьютеризации.

Процесс компьютеризации – продуктивная деятельность человека или группы, целенаправленно осуществляемая в рамках объекта компьютеризации и имеющая целью достижение конкретного результата, представляющего ценность для потребителя.

Объект компьютеризации – область деятельности человека или группы, характеризующаяся определенной целью и обладающая свойствами системы, элементами которой являются в том числе люди как субъекты деятельности.

Пример формулировки задания на разработку ПС на базе заданного объекта и процесса компьютеризации:

Объект компьютеризации: торговое предприятие.

Процесс компьютеризации: складской учет товаров.

Соответствующая тема: Программная система складского учета товаров.

Пример формулировки объекта и процесса компьютеризации на базе задания на разработку ПС:

Соответствующая тема: бортовая система идентификация водителя автомобиля по изображению лица.

Процесс компьютеризации: идентификация водителя автомобиля по изображению лица.

Объект компьютеризации: автомобиль.

Объект компьютеризации описывается лаконично простым повествовательным техническим текстом и содержит такие сведения:

1. Наименование объекта. Если объектом является организация, указывается ее юридическое название. Если объектом является техническое устройство, указывается его название согласно официальной документации на него.

2. Краткая характеристика объекта в 3–5 простых предложениях, раскрывающая его суть. Здесь уместно дать ссылку на источник этой информации.

3. Структурная схема, иллюстрирующая архитектуру объекта. Если объектом является организация, приводится её организационная диаграмма. Если объектом является техническое устройство, приводится его функциональная структура или алгоритм работы. Уместны и другие иллюстрации, если они значимо дополняют текстовую часть описания.

4. Текстовые комментарии компонентов структурной схемы, в том числе задачи (функции) компонентов и особенности их взаимодействия при функционировании объекта.

5. Основные показатели эффективности функционирования объекта (не более трех), на улучшение которых должна быть направлена компьютеризация.

Пример исчерпывающего описания объекта представлен в [приложении Б](#).

Процесс компьютеризации, как правило, выбирается среди функционала объекта компьютеризации. Укрупнено его можно представить в нотации IDEF0 в виде модели «Черный ящик».

При системном анализе необходимо определить целевую функцию – результат работы изучаемой системы (например, для объекта «шахта» целевая функция – добыча угля, для объекта «автомобиль» – движение по автодорогам).

Если объектом является организация, то процесс компьютеризации представляет собой *бизнес-процесс*. Если объект технический, то процесс компьютеризации представляет собой *технологический процесс*. В любом случае процесс компьютеризации объединяет в себе несколько задач (функций) объекта. Такую структуру можно представить в виде диаграммы декомпозиции.

Разделение бизнес-процесса на подпроцессы (задачи) производится с учетом целевой функции бизнес-процесса, а деление операции – с учетом целевой функции подпроцесса, исходя из целевой функции бизнес-процесса. Такой принцип упрощает изучение сложных бизнес-процессов и является системным подходом.

В каждом конкретном случае процессу компьютеризации дается лаконичное и информативное наименование, в полной мере отражающее суть выполняемых действий на объекте. Именованые задачи и операций также выполняются в соответствии с этим принципом.

При выделении задач следует помнить, что задача программы – это формализованная совокупность действий, выполнение которых приводит к результату заданного вида. Поэтому в качестве задач надо выбирать такие, для

которых можно четко сформулировать результат. В данном примере можно выделить такие задачи:

- задача ввода входных данных;
- задача сохранения данных в памяти ЭВМ;
- задача формирования выходных данных;
- задача вычисления некоторого итогового показателя;
- задача статистического анализа данных, и др.

1.3. Описание информационных потоков

При выделении задач следует учитывать, что это информационно взаимосвязанный функциональный комплекс, назначение которого – переработка входной информации в выходную. Любой функциональный элемент, будь то процесс, задача или операция, получает на вход некоторую входную информацию (исходные данные, задание) и генерирует некоторую выходную результатную информацию. Возможно также выделение внутренней информации, циркулирующая внутри процесса. Как правило, это промежуточные данные, являющиеся результатами переработки входных инфопотоков и исходными данными для выработки выходных инфопотоков.

Вся информация циркулирует в рамках процесса компьютеризации в форме информационных потоков и требует описания и систематизации. Такое описание уместно представить форме реестров (табл. 1, 2).

Таблица 1. Реестр входных информационных потоков

№	Наименование и назначение потока	Форма представления	Обработчик (Кто обрабатывает)	Корреспондент (Откуда поступает)	Характеристики обработки	
					Трудозатраты, чел.ч	Периодичность, регламент

Таблица 2. Реестр выходных информационных потоков

№	Наименование и назначение потока	Форма представления	Обработчик (Кто обрабатывает)	Корреспондент (Куда отправляется)	Характеристики обработки	
					Трудозатраты, чел.ч	Периодичность, регламент

Возможные формы представления информации: сигнал, сообщение, устное распоряжение, документ, датасет, и др. Наименование потока должно адекватно отражать его сущность и соотноситься с его формой представления. Если формой представления инфопотока является документ, целесообразно представить его макет, либо копию. При этом необходимо кратко описать такие реквизиты документа, смысл которых не является очевидным.

Трудозатраты представляют собой вещественную неотрицательную оценку, выражающую количество часов, которые должен работать 1 человек, чтобы воспринять и обработать входной поток или сформировать и вывести выходной поток. В редких случаях, когда человек не причастен к обработке инфопотока, выставляется оценка 0.

Периодичность отражает частоту следования инфопотока. Как правило, даются средние оценки. Наилучший способ определения этой характеристики – получить экспертное мнение специалиста из предметной области. Если такого специалиста нет, то периодичность следует оценить исходя из назначения инфопотока и логики его обработки. К примеру, для оперативной информации периодичность может составлять от 10 раз в сутки (например, заявка на ремонт оборудования) до 100 раз в секунду (например, сигнал от датчика системы реального времени). Для справочной информации периодичность гораздо ниже и может составлять 1 раз в год (обновление штатного расписания) или 10 раз в месяц (сведения о трудоустраиваемом лице). В крайнем случае, если никак не возможно установить приблизительную частоту следования инфопотока, допускается указать «По требованию».

Если формой представления инфопотока является документ, целесообразно представить его макет, либо копию. При этом необходимо кратко описать такие реквизиты документа, смысл которых не является очевидным.

1.4. Постановка задачи на создание ПС

Постановка задачи на создание ПС, как правило, сводится к разработке технического задания (ТЗ).

Техническое задание на создание ПС – основной документ, определяющий требования и порядок создания (развития или модернизации) ПС, в соответствии с которым проводится разработка ПС и ее приемка при вводе в действие.

ТЗ разрабатывают на систему в целом, предназначенную для работы самостоятельно или в составе другой системы. Этот документ, которым руководствуются разработчики и проектировщики, приступая к разработке нового продукта, определяет основные направления разработки: технологии и принципы работы будущего продукта.

Техническое задание преследует множество целей. Среди них:

- постановка задачи исполнителям;
- подробное описание конечного программного продукта;
- определение порядка работ и его согласование с заказчиком;
- определение процедуры оценки приема конечного программного продукта.

Требования к ПС во многом аналогичны требованиям к ПО. Вместе с тем ТЗ на создание системы включает в себя также требования к её обеспечивающим подсистемам (декомпозиция ПС на функциональные и обеспечивающие средства будет рассмотрена в параграфе 1.2). Эти требования также относятся к категории нефункциональных требований. Выделяют четыре основных обеспечивающих подсистемы: информационное, математическое, программное и техническое обеспечение ПС.

Требования к информационному обеспечению могут включать следующие пункты:

- требования к организации данных, которые должны сохраняться в ПС;
- отсутствие дублирования информации и сокращение чрезмерности данных, поддержка целостности, достоверности и актуальности данных;
- сетевой режим доступа к общим данным, распределение информационных ресурсов в подсистеме;
- низкая стоимость хранения и использования данных;
- возможность получения данных с помощью языка запросов высокого уровня без использования прикладных программ;
- защита от несанкционированного доступа к данным, их порчи и уничтожения;
- обеспечение конфиденциальности секретной информации;
- возможность ведения архивов и восстановление данных в случае разрушения баз данных после сбоев.

Для математического обеспечения приводят требования к составу, области применения (ограничения) и средств использования в системе математических методов и моделей, типичных алгоритмов и алгоритмов, которые подлежат разработке.

В требованиях к программному обеспечению ПС указываются:

- требования к структуре ПО подсистемы;
- требования к операционной среде;
- требования к инструментальным средствам разработки ПО;
- требования к использованию готовых программных пакетов;
- требования к составу и функциям специального ПО, подлежащему разработке;
- требования к вспомогательным программным средствам (сервисные программы и утилиты).

В требованиях к техническому обеспечению указываются требования к функциональным, конструктивным и эксплуатационным характеристикам технических средств ПС, а именно требования относительно выбора не менее двух конфигураций ЭВМ: для сервера (серверов) и клиентских рабочих станций. При этом необходимо учитывать, что эти требования обусловлены спецификой задач, которые решаются с помощью ПС.

Задача ПС – функция ПС или часть функции ПС, являющаяся формализованной совокупностью действий, выполнение которых приводит к результату заданного вида (для которых можно четко сформулировать результат). Например, задача ввода входных данных, задача формирования статистической отчетности, задача диагностики состояния объекта, задача анализа данных и принятия решений и др.

При описании требований к аппаратуре необходимо указывать не конкретные значения характеристик аппаратных компонентов или конкретные типы, а лишь определенные ограничения на эти характеристики. Например, не следует писать «...основная память должна поддерживать частоту шины N ГГц,

... должна быть построена на модулях типа ...». Правильней здесь могла быть фраза «...частота, поддерживаемая основной памятью компьютера, должна быть не ниже N ГГц...» с последующим обоснованием.

Рекомендуемая литература

- [1] – главы 2 и 3.
- [2] – разделы 1.1, 1.4, подразделы 1.1.8, 1.2.6, 1.3.5 и 2.1.6.
- [3] – главы 1–3, разделы 4.1–4.5.
- [5] – главы 6, 11, 12.
- [7] – разделы 4.1–4.4.

Контрольные вопросы

1. Что такое эскизный проект и какова его структура?
2. Что такое объект и процесс компьютеризации? Как они соотносятся?
3. В чем заключается принцип системного анализа? Что такое целевая функция системы (приведите примеры)?
4. Насколько широк спектр существующих автоматизированных систем, которые уместно применить к выбранному объекту компьютеризации?
5. Какая из существующих автоматизированных систем наиболее близка для применения на выбранном объекте компьютеризации, и почему?
6. Что такое техническое задание и какова его структура?
7. Какие положительные результаты могут быть получены в процессе использования ПС?
8. Каково назначение разрабатываемой ПС?
9. Назовите цели, в соответствии с которыми разрабатывается ПС.
10. Что такое функциональные и нефункциональные требования к ПС?
11. Какие виды обеспечения ПС разрабатываются при ее создании?
12. Какие требования предъявляются к входным и выходным данным ПС?

Лабораторная работа 2

Тема: проектирование функциональной структуры ПС

Цель: освоение методики разработки и описания функциональной части разрабатываемой ПС, закрепление навыков применения методик функционально-ориентированного и объектно-ориентированного подходов программной инженерии для разработки и описания функциональности ПС.

Задание

1. Обосновать создание разрабатываемой ПС и ее структурную организацию.
2. Выполнить описание процесса функционирования объекта компьютеризации в условиях ПС.
3. Выделить и описать автоматизированные функции, исполняемые подсистемой.
4. На основе результатов, полученных при выполнении пп. 1–3, построить комплексную функциональную модель разрабатываемой ПС.

Требования к содержанию отчета

В подразделе 1 отражаются необходимые изменения в процессе, который автоматизируется, с учетом внедрения ПС. Описывается и обосновывается структура ПС. Если ПС включает в себя несколько подсистем, выполняется краткое описание каждой подсистемы ПС.

В подразделе 2 приводится описание процесса функционирования объекта автоматизации в условиях функционирования ПС. Составляется описание процессов выполнения каждой задачи в соответствии с требованиями, которые были сформированы в техническом задании ([лабораторная работа 1](#)). Выделяются существенные структурные элементы и действия, определяются связи между ними.

В подразделе 3 по пунктам перечисляются автоматизированные функции разрабатываемой ПС. По каждой функции выполняется описание входных данных и получаемых результатов. Отдельно рассматриваются действия (операции), которые выполняются программными или техническими средствами, и человеком. Отражаются все действия ПС, направленные на компьютерную обработку и хранение входной и выработку выходной информации.

По результатам подразделов 1–3 строится комплексная функциональная модель ПС в виде иерархии диаграмм IDEF0 и диаграмм поведения UML.

В ФОП при построении контекстной диаграммы ПС за основу принимается разработанная при выполнении [лабораторной работы 1](#) функциональная схема типа «черный ящик» с обозначенными входными (Input) выходными (Output) данными, а также списков нормативно-справочной документации (Control) и экторов, задействованных в бизнес-процессе (Mechanism).

Следует иметь в виду, что при выполнении [лабораторной работы 1](#) моделью черного ящика был описан процесс компьютеризации как есть. На этапе постановки задачи происходит переосмысление процесса компьютеризации и некоторые его задачи проектируемой ПС реализуются полностью автоматически. В таком случае внешнее дополнение ПС на модели черного ящика может отличаться от исходной модели процесса компьютеризации.

При реализации ООП, основываясь на описаниях функционирования ПС, полученных при выполнении пп. 2 и 3 и опираясь на диаграмму требований UML, построенную при выполнении [лабораторной работы 1](#), необходимо построить полную диаграмму классов. Также необходимо построить комплекс диаграмм поведения, в достаточной степени раскрывающих функциональность всех методов классов, классов и особенности их взаимодействия с другими классами и действующими лицами, которые представлены на диаграмме требований UML.

Все диаграммы и их логические и структурные взаимосвязи должны быть описаны по тексту отчета.

Рекомендуемая литература

- [1] – раздел 1.2.
- [2] – подразделы 1.1.2 и 1.1.3.
- [3] – глава 3.

Контрольные вопросы

1. Перечислить и охарактеризовать автоматизированные функции разрабатываемой ПС.
2. С какой периодичностью выполняется какая-либо функция ПС?
3. Что такое функционально-структурная схема ПС?
4. Какие функции возлагаются на человека как элемента в структуре разрабатываемой ПС?
5. Какова численность оперативного персонала, задействованного в автоматизируемых задачах до и после предполагаемого внедрения ПС?
6. Перечислить объекты, на которых целесообразна эксплуатация ПС?
7. В чем заключается функционально-ориентированный подход к разработке и описанию функциональности компьютерной программы?
8. Каково назначение и структура функциональной модели компьютерной программы?
9. Что такое перечисление автоматизированных функций компьютерной программы?
10. Чем отличаются модели черного ящика, построенные при выполнении лабораторных работ 1 и 2?
11. В чем заключается метод объектно-ориентированного анализа?

12. В чем заключается метод объектно-ориентированного проектирования?

13. Как соотносятся между собой базовые диаграммы UML: диаграмма требований и диаграмма классов?

14. Как соотносятся между собой базовые диаграммы UML и диаграммы в нотации IDEF0?

15. Какие CASE-средства могут быть использованы для объектно-ориентированного проектирования ПО?

16. Каково назначение диаграмм поведения UML?

17. Как взаимосвязаны диаграммы поведения UML и базовые диаграммы UML?

18. Откуда берутся исходные данные для построения диаграммы деятельности UML?

19. Откуда берутся исходные данные для построения диаграмм взаимодействия UML?

20. Какие диаграммы UML входят в модель программного продукта на языке UML?

21. Сколько и каких диаграмм поведения должно быть построено, если диаграмма требований содержит состоит три базовых прецедента?

Лабораторная работа 3

Тема: формирование среды разработки ПС.

Цель: освоение технологии конфигурационного управления программным процессом, развертывания рабочей среды для разработки ПС, настройки и конфигурирования рабочей среды, создание репозитория для контроля версий ПС.

Задание

1. В соответствии с требованиями технического задания, разработанного при выполнении [лабораторной работы 1](#), провести обоснованный выбор средства управления данными и средства разработки моделей данных.
2. В соответствии с требованиями технического задания, разработанного при выполнении [лабораторной работы 1](#), провести обоснованный выбор средств разработки специального ПО.
3. Создать удаленный репозиторий git (например, на github.com) с названием по имени проекта или по фамилии студента, создать инициализирующий коммит. При необходимости добавить к удаленному репозиторию участников проекта.
4. Провести идентификацию инструментальных и организационных рисков проекта.

Требования к содержанию отчета

В подразделе 1 студент определяет способ представления данных, которые будут сохраняться в ПС. Как правило, делается сравнения разных даталогических моделей и выбор реляционной требуемой (например, реляционной) модели данных. Далее проводится сравнения (например, в виде таблицы) двух современных систем управления базами данных (СУБД) серверного типа приблизительно одинаковой мощности. Проводится обоснования выбора одной из них как средства управления данными в ПС. При необходимости ведется выбор отдельной СУБД для клиентской части приложения или для отдельных сервисов при использовании микросервисной архитектуры.

Также производится обоснованный выбор инструментального средства для разработки логической и физической модели данных. Для этой цели можно использовать UML или рекомендуемое CASE-средство ERWin. Необходимо привести обоснование выбора средства разработки модели данных с учетом выбранной СУБД.

Объем подраздела не более одной страницы.

В подразделе 2 проводится анализ и обоснованный выбор программных средств для разработки специального ПО, то есть для создания программных модулей, реализующих автоматизированные функции ПС, и сборки ПС в целом. Для анализа выбирается не менее 2-х современных систем (языков)

программирования, платформ разработки и сопровождения ПО, средство контейнеризации и оркестрации, а также управления жизненным циклом ПО (при необходимости), которые пригодны для разработки соответствующей ПС. Для каждого средства указываются основные возможности и характерные особенности без подробного описания языка программирования, интегрированной среды и т. п. В результате анализа делается вывод, на основании чего выбираются средства разработки специального ПО.

Объем подраздела не более двух страниц.

Выбранные в подразделах 1 и 2 инструменты должны соответствовать требованиям к инструментам, предъявленным в техническом задании на создание ПС. При необходимости следует скорректировать ТЗ.

В подразделе 3 выполняется создание репозитория проекта ПС для осуществления версионного контроля проекта. Может быть использована любая система контроля версий (Microsoft SourceSafe, IBM ClearCase, CVS, Subversion, Perforce, Git, Mercurial и др.). Рекомендуемая система управления версиями – Subversion или Git. Сформировать репозиторий приложения:

- при использовании Subversion – при помощи клиента TortoiseSVN;
- при использовании Git – при помощи веб-сервиса GitHub.

В отчете представить скриншот развернутого репозитория и открытую ссылку на созданный проект.

В подразделе 4 выполняется первый этап управления рисками программного процесса. Основываясь на материалах подразделов 1–3 отчета, а также с учетом результатов выполнения лабораторных работ 1 и 2 – планирования ресурсов проекта и проектных решений по функциональной структуре ПС соответственно, необходимо идентифицировать риски:

1) инструментально-технологические (категория Т), связанные с выбором и применением инструментального ПО;

2) организационные (категория О), связанные с организацией работ по проекту, включая обеспечение ресурсами и другие условия работ (см. рис. 1).

Формулировки рисков и их последствий должны быть конкретными. Следует избегать общих формулировок, соотнося их со спецификой проекта.

Результаты идентификации внести в таблицу:

Шифр	Риск	Последствия риска
Т-1		
О-1		
...		

Рекомендуемая литература

[1] – главы 4, 7.

[2] – разделы 1.4, 2.2, подразделы 1.2.4, 1.2.8.

[3] – разделы 5.2, 5.4.

[4] – подраздел 1.2.4, раздел 2.2.

[5] – глава 10.



Рисунок 1. Условия проектной деятельности

Контрольные вопросы

1. Что такое программное обеспечение ПС?
2. Что такое система управления версиями?
3. Как создать репозиторий?
4. Как создать ветку?
5. Как провести слияние? Как разрешить конфликт и что это такое?
6. Как зафиксировать изменения?
7. Как провести откат? Различия в reset и revert, мягкий и жесткий reset.
8. Какова последовательность действий при работе с удаленным репозиторием?
9. Каковы возможности при работе с удаленным репозиторием? Как его клонировать, получать и отправлять данные?
10. Основные команды git.
11. Как работают команды reset, revert, rebase, stash, tag?
12. Как создавать и применять изменения (patch)? Что это такое?
13. Что такое риски? Какие бывают риски в сфере разработки ПС?
14. В чем заключается этап идентификации рисков?
15. Перечислите и охарактеризуйте условия проектной деятельности, определяющие рискованный фон проекта ПС.

Лабораторная работа 4

Тема: разработка информационного обеспечения ПС.

Цель: изучение программных средств для разработки моделей информационной базы ПС, проработка методов нормализации отношений в БД, приобретение навыков применения CASE-средства ERwin для моделирования базы данных ПС.

Задание

1. На основе диаграммы классов UML, разработанной при выполнении [лабораторной работы 2](#), произвести идентификацию сущностей информационной базы ПС и связей между ними.

2. При помощи CASE-средства ERWin разработать ER-диаграмму логической модели данных по принципу: один класс – одна сущность.

3. В соответствии с требованиями технического задания, разработанного при выполнении [лабораторной работы 1](#), провести обоснованный выбор СУБД.

4. Провести нормализацию сущностей логической модели данных и при помощи CASE-средства ERWin разработать ER-диаграмму физической модели данных с учетом выбранной СУБД.

5. Средствами ERWin на основе физической модели данных выполнить генерацию SQL-кода для создания реляционной базы данных ПС.

6. В выбранной СУБД развернуть БД, доработать её структуру с учетом возможной нормализации отношений, а также доработать структуру таблиц с учетом ограничений на значения полей. **В отчете представить изображение, подтверждающее факт развертывания БД.**

7. Выполнить описание таблиц БД в формате табл. 3 с указанием для всех полей типов данных выбранной СУБД. Столбцы «Условие на значение», «Значение по умолчанию» и «Примечание» заполнять при необходимости. Выполнить краткое описание таблиц БД и связей между ними в формате табл. 4 и 5.

Таблица 3. Структура таблицы

Имя поля	Тип данных	Размер, байт	Условие на значение*	Значение по умолчанию*	Примечание*
----------	------------	--------------	----------------------	------------------------	-------------

* Указанные столбцы не являются обязательными для заполнения

Таблица 4. Список разработанных таблиц

№ п/п	Имя таблицы	Описание
-------	-------------	----------

Таблица 5. Связи между таблицами БД

Родительская таблица		Дочерняя таблица		Тип связи*
Название	Атрибут РК	Название	Атрибут FK	

* в качестве типа связи указывать «1:1», «1:M».

Теоретические положения

4.1. CASE-средство ERwin для моделирования баз данных

Разработка ПС с использованием CASE-технологий является наиболее эффективным методом и имеет массу преимуществ, как для пользователей, так и для разработчиков. Для последних CASE-средства вместе с системным ПО и техническими средствами образуют полную среду разработки ПС.

Одним из основных поставщиков CASE-средств служит американская компания CA Technologies, которая является разработчиком программных систем и решений управления ИТ-средой, который обладает опытом работы со всеми ИТ-средами, от больших ЭВМ и распределенных сред до виртуальных и облачных сред. Данная компания занимается разработкой CASE-средств BPwin Process Modeler и ERwin Data Modeler, которое мы будем использовать при создании базы данных ПС.

ERwin Data Modeler (или просто [ERwin](#)) – CASE-средство для проектирования и документирования баз данных, которое позволяет создавать, документировать и сопровождать базы данных, хранилища и витрины данных. Модели данных помогают визуализировать структуру данных, обеспечивая эффективный процесс организации, управления и администрирования таких аспектов деятельности предприятия, как уровень сложности данных, технологий баз данных и среды развертывания.

ERwin предназначен для всех компаний, разрабатывающих и использующих базы данных, для администраторов баз данных, системных аналитиков, проектировщиков баз данных, разработчиков, руководителей проектов. ERwin Data Modeler позволяет управлять данными в процессе корпоративных изменений, а также в условиях стремительно изменяющихся технологий.

ERwin позволяет наглядно отображать сложные структуры данных. Удобная в использовании графическая среда ERwin Data Modeler упрощает разработку базы данных и автоматизирует множество трудоемких задач, уменьшая сроки создания высококачественных и высокопроизводительных транзакционных баз данных и хранилищ данных.

ERwin имеет мощные средства визуализации модели, такие, как использование различных шрифтов, цветов и отображение модели на различных уровнях, например, на уровне описания сущности, на уровне первичных ключей сущности и т.д. Эти средства ERwin значительно помогают при презентации модели в кругу разработчиков системы или сторонним лицам.

Обычно разработка модели базы данных состоит из двух этапов: составление логической модели и создание на ее основе физической модели. ERwin полностью поддерживает такой процесс, он имеет два представления модели: логическое (logical) и физическое (physical). Таким образом, разработчик может строить логическую модель базы данных, не задумываясь над деталями физической реализации, т.е. уделяя основное внимание

требованиям к информации и бизнес-процессам, которые будет поддерживать будущая база данных.

Далее при помощи ERwin разработаем логическую модель данных, и создадим на ее основе физическую модель.

4.2. Разработка логической модели данных

Логическая модель описывает понятия предметной области, их взаимосвязь, а также ограничения на данные, налагаемые предметной областью. Логическая модель данных является визуальным графическим представлением структур данных, их атрибутов и связей. Логическая модель представляет данные таким образом, чтобы они легко воспринимались бизнес-пользователями.

Логическая модель данных является начальным прототипом будущей базы данных. Она строится в терминах информационных единиц, но без привязки к конкретной СУБД.

Основными компонентами логической модели являются:

- сущности;
- атрибуты сущности;
- связи между сущностями.

Сущность моделирует структуру однотипных информационных объектов (документов, хранилищ данных, таблиц базы данных). В пределах модели данных сущность имеет уникальное имя, выраженное существительным.

Сущность является шаблоном, на основании которого создаются конкретные экземпляры сущности.

Сущность обладает следующими свойствами:

- каждая сущность имеет уникальное имя, и к одному и тому же имени должна применяться одинаковая интерпретация;
- сущность обладает одним или несколькими атрибутами, которые либо принадлежат сущности либо наследуются через связь;
- сущность обладает одним или несколькими атрибутами, которые однозначно идентифицируют каждый экземпляр сущности;
- каждая сущность может обладать любым количеством связей с другими сущностями модели.

Атрибут – любая характеристика сущности, значимая для рассматриваемой предметной области и предназначенная для квалификации, идентификации, классификации, количественной характеристики или выражения состояния сущности. Атрибут представляет тип характеристик или свойств, ассоциированных со множеством реальных или абстрактных объектов (людей, мест, событий, состояний, идей, пар предметов и т.д.). Экземпляр атрибута – это определенная характеристика отдельного элемента множества. Экземпляр атрибута определяется типом характеристики и ее значением, называемым значением атрибута. В ER-модели атрибуты ассоциируются с конкретными сущностями. Таким образом, экземпляр сущности должен

обладать единственным определенным значением для ассоциированного атрибута.

Связь – поименованная ассоциация между двумя сущностями, значимая для рассматриваемой предметной области. Связь – это ассоциация между сущностями, при которой, как правило, каждый экземпляр одной сущности, называемой родительской сущностью, ассоциирован с произвольным (в том числе нулевым) количеством экземпляров второй сущности, называемой сущностью-потомком, а каждый экземпляр сущности-потомка ассоциирован в точности с одним экземпляром сущности-родителя.

На примере бизнес-процесса управления продажами книг произведем идентификацию сущностей ПС и связей между ними, приведенных в табл. 6.

Таблица 6. Идентификация сущностей (пример заполнения)

Документ	Сущность
Накладные	Накладные
Счет-фактура	Счета
Книги	Товары
Клиенты	Клиенты
Заказы	Заказы

Исходя из данных табл. 6, каждой сущности в соответствие ставится документ. На данном этапе получаем пять основных сущностей, которые далее, при разработке физической модели данных, разобьем на подсущности.

Далее в табл. 7 приведем имена, атрибуты и назначения сущностей.

Таблица 7. Сущности логической модели данных

Сущность	Атрибуты	Описание
Справочные		
1 Товары	<u>Код товара</u> , название книги, жанр, страницы, год издания, код автора, фамилия, имя, отчество, код издательства, название издательства, город сокращенно, город полностью	Информация о книгах авторов и издательствах
2 Клиенты	<u>Код клиента</u> , Фамилия, Имя, Отчество, телефон	Информация о клиентах
Оперативные		
3 Заказы	<u>Код заказа</u> , <i>Код товара</i> , количество, сумма, код поставщика, имя поставщика, номер телефона, адрес	Информация по заказам и поставщикам
4 Накладные	<u>Код накладной</u> , номер накладной, дата создания, вид накладной, цена, сумма, <i>код счета</i> , <i>код заказа</i> , <i>код клиента</i> , <i>код товара</i> , код сотрудника, фамилия, имя, отчество	Информация о накладных и сотрудниках
5 Счета	<u>Код счет-фактуры</u> , дата создания, <i>код заказа</i> , <i>код товара</i> , <i>код клиента</i> , количество, цена	Информация о счетах

Сущности в табл. 7 разделим на оперативные и справочные. Справочные сущности – это сущности, информация о которых хранится в справочниках.

Как правило, это неизменная и многократно используемая в течение длительного периода времени информация. Информация об оперативных сущностях может меняться для каждого случая, как по назначению, так и по количеству. При разработке модели определим сущности, их первичные и внешние ключи и атрибуты, а также связи между сущностями.

При помощи CASE-средства ERWin разработаем ER-диаграмму логической модели данных на основе табл. 7, представленную на рис. 2.

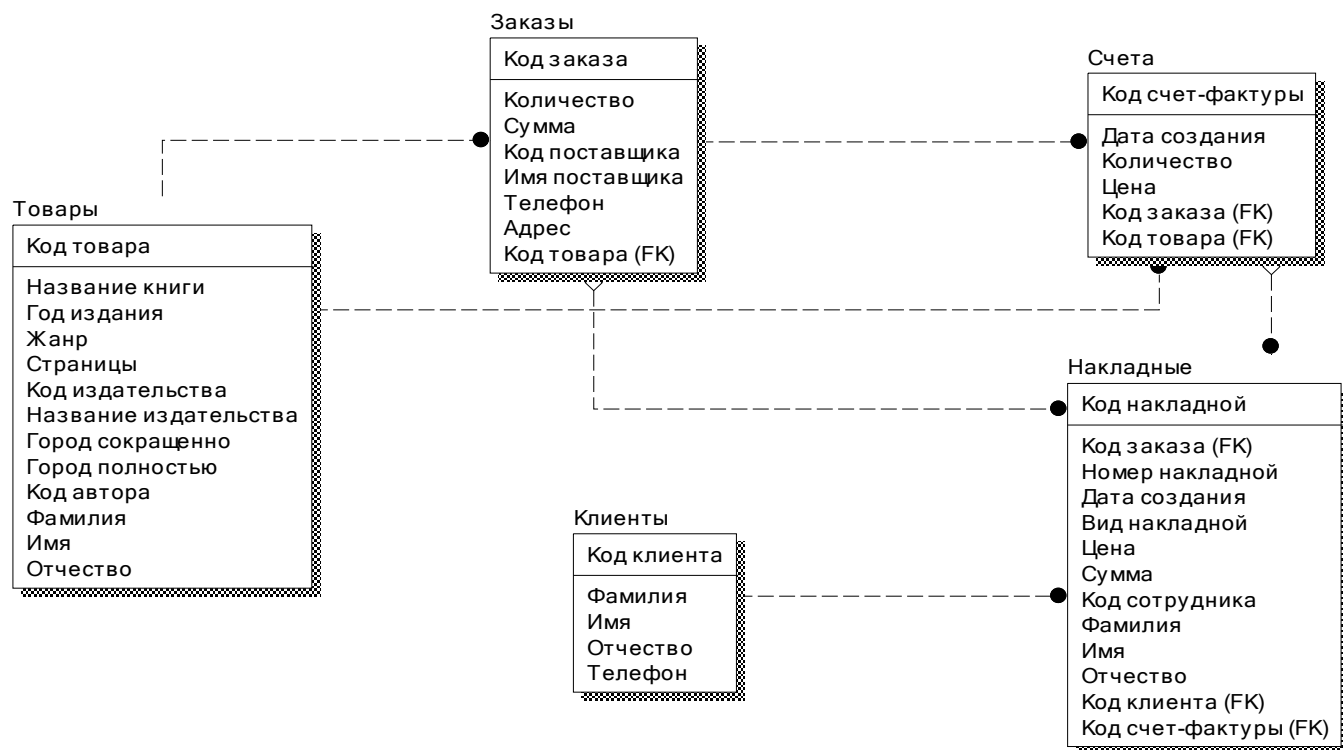


Рисунок 2. ER-диаграмма логической модели данных

4.3. Разработка физической модели данных

Физическая модель баз данных определяет способы размещения данных в среде хранения и способы доступа к этим данным, которые поддерживаются на физическом уровне, является графическим представлением реально реализованной базы данных. В физической модели содержится информация обо всех объектах БД. Поскольку стандартов на объекты БД не существует (например, нет стандарта на типы данных), физическая модель зависит от конкретной реализации СУБД. Физическая база данных будет состоять из таблиц, столбцов и связей. Физическая модель зависит от платформы, выбранной для реализации, и требований к использованию данных. Компонентами физической модели данных являются таблицы, столбцы и отношения. Сущности логической модели, вероятно, станут таблицами в физической модели. Логические атрибуты станут столбцами. Логические отношения станут ограничениями целостности связей.

На основе логической модели данных разработаем при помощи CASE-средства ERWin ER-диаграмму физической модели данных. В ERWin необходимо выбрать тип модели (рис. 3).

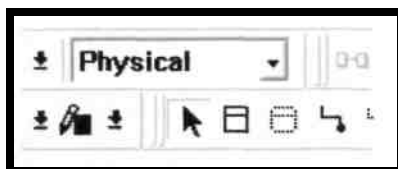


Рисунок 3. Выбор типа модели данных в ERWin

Для этого выделим из сущностей подсущности для нормализации отношений. Отношения, разработанные на стадии формирования логической модели данных, преобразуются в таблицы, атрибуты становятся столбцами таблиц, для ключевых атрибутов создаются уникальные индексы, домены преобразуются в типы данных, принятые в конкретной СУБД.

Ограничения, имеющиеся в логической модели данных, реализуются различными средствами СУБД, например, при помощи индексов, декларативных ограничений целостности, триггеров, хранимых процедур. При этом опять-таки решения, принятые на уровне логического моделирования определяют некоторые границы, в пределах которых можно развивать физическую модель данных. Точно также, в пределах этих границ можно принимать различные решения. Например, отношения, содержащиеся в логической модели данных, должны быть преобразованы в таблицы, но для каждой таблицы можно дополнительно объявить различные индексы, повышающие скорость обращения к данным. Многое тут зависит от конкретной СУБД.

Из сущности Товары выделяем подсущности Авторы и Издательства; из сущности Заказы выделяем Поставщиков и Позиции заказа; из сущности Накладные выделяем Позиции, Виды накладных и Сотрудников; из сущности Счета выделяем Позиции счета; сущность Клиенты оставляем неизменной. Результатная модель показана на рис. 4.

Далее средствами ERWin (см. рис. 5) выполним генерацию SQL-кода для создания реляционной базы данных ПС. Алгоритм действий следующий:

- при помощи CASE-средства ER-Win открыть физическую модель и вызвать диалог генерации SQL-кода (меню Tools → Forward Engineer → Schema Generation...);
- выбрать формат будущей БД (SQL Server, Interbase, Oracle, и т.д.). Лучше отключить генерацию триггеров, для чего в группе триггер posнимать все птички;
- нажать на кнопку Preview...;
- в появившемся диалоговом окне скопировать сгенерированный код в буфер обмена.

Формируется список разработанных таблиц и связей между таблицами БД, описание сущностей физической модели в формате табл. 3.

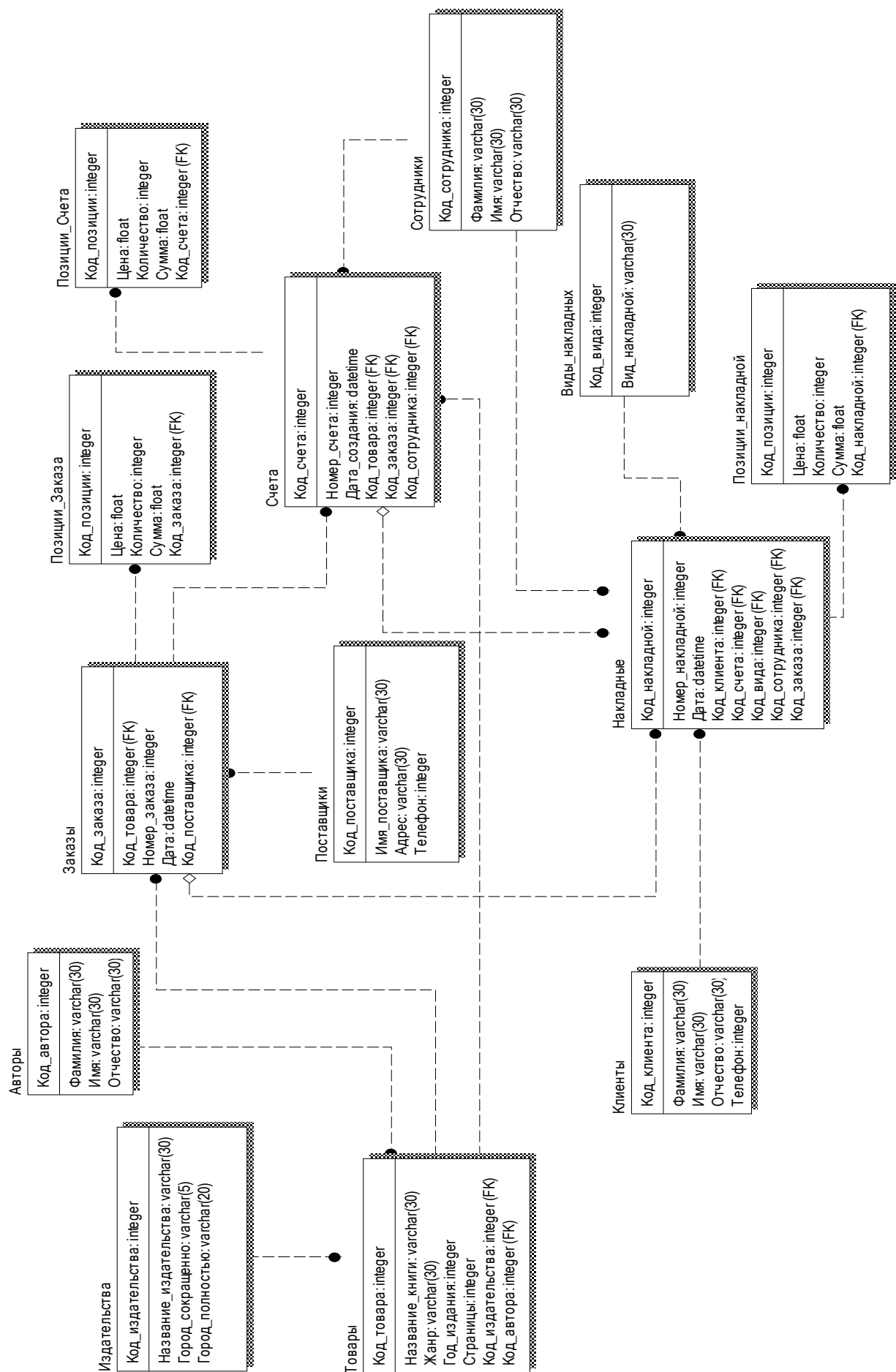
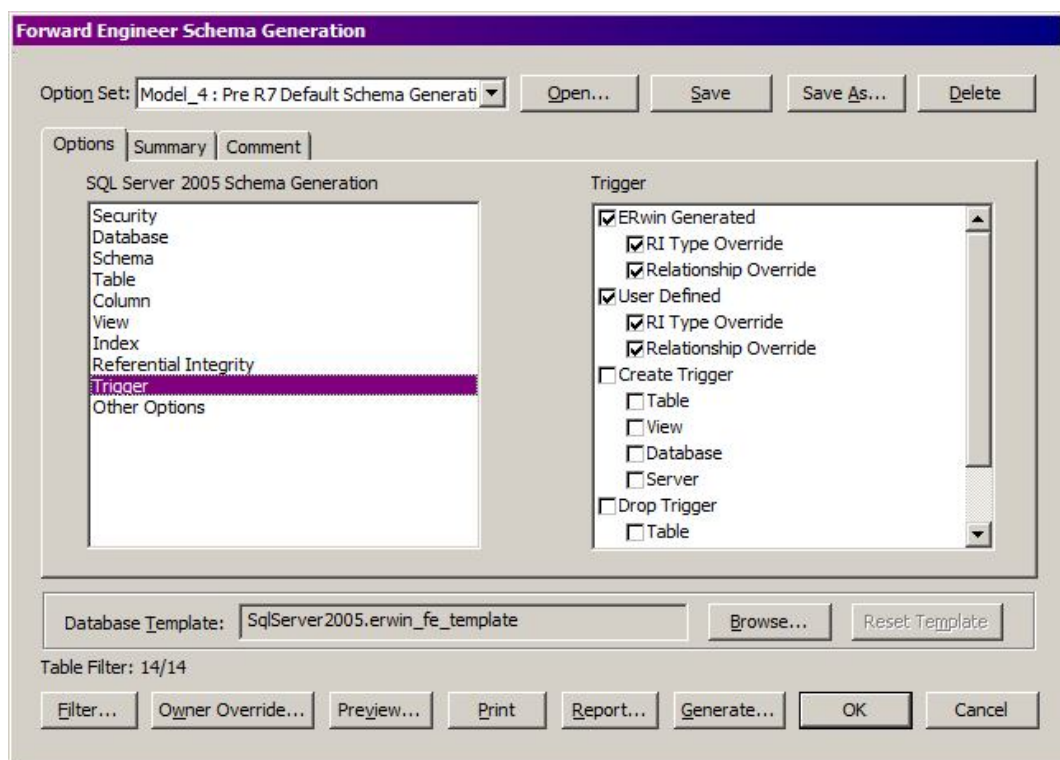
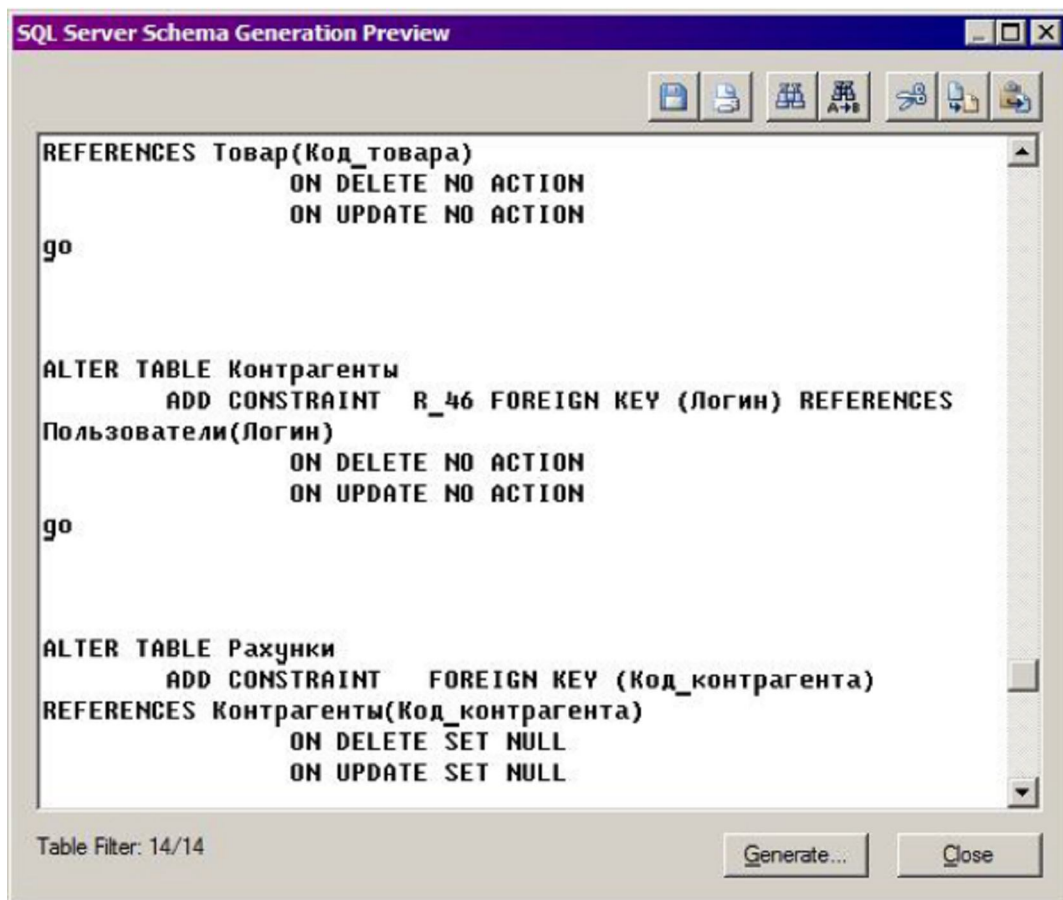


Рисунок 4. ER-диаграмма физической модели данных



a



б

Рисунок 5. Настройка и результаты процедуры генерации кода SQL:
а – генератор кода SQL; б – результат генерации

Рекомендуемая литература

- [1] – раздел 1.6.
- [2] – разделы 1.4, 2.2, подразделы 1.2.4, 2.1.8
- [3] – разделы 5.2, 6.1.
- [4] – подраздел 1.2.4.

Контрольные вопросы

1. Что такое информационное обеспечение ПС?
2. Что такое информационная база ПС?
3. Опишите алгоритм создания информационной базы ПС?
4. Какие современные СУБД используются для создания информационной базы ПС?
5. Что такое нормализация отношений БД?
6. Чем отличается физическая модель данных от логической?
7. В чем различия между справочными таблицами и таблицами оперативных данных?

Лабораторная работа 5

Тема: разработка программного обеспечения ПС

Цель: приобретение навыков проектирования и разработки программного обеспечения ПС при помощи современных инструментальных средств, а также его тестирования методом черного ящика

Задание

1. В соответствии с требованиями технического задания, разработанного при выполнении [лабораторной работы 1](#), и решениями по выбору средств разработки специального ПО программной системы, принятыми при выполнении [лабораторной работы 3](#), выбрать системные, служебные и прикладные программы, необходимые для разработки и эксплуатации ПС. Также определить укрупненные модули специального ПО. Разработать схему ПО программной системы на подобие схемы, показанной на рис. 6.

2. В соответствии с требованиями технического задания, разработанного при выполнении лабораторной работы 1, а также проектными решениями, разработанными при выполнении лабораторных работ 2 и 4, разработать специальное программное обеспечение ПС.

3. Выполнить описание разработанных компонентов приложения в виде табл. 8. Типы компонентов указать согласно табл. 9. Имена компонентов-файлов привести с указанием расширения.

Таблица 8. Перечень разработанных компонентов приложения

№	Имя	Тип	Описание
---	-----	-----	----------

Таблица 9. Типы компонентов приложения

Стереотип	Тип компонента
<<source>>	Файл исходного текста программы
<<form>>	Файл интерфейсной формы или web-страницы
<<executable>>	Исполняемый файл
<<config>>	Файл конфигурации и/или настроек системы
<<library>>	Статическая или динамическая библиотека
<<document>>	Файл документации
<<data>>	Файл данных
<<table>>	Таблица базы данных
<<database>>	База данных
<<view>>	Представление как объект базы данных
<<file>>	Файл, не попадающий под тип, указанный выше в этой таблице

4. Построить структурную схему разработанного приложения в виде диаграммы компонентов UML, выражающую взаимодействие его компонентов с компонентами БД в процессе функционирования приложения.

5. Провести модульное тестирование компонентов приложения. Убедиться в соответствии результатов работы компонентов требованиям, установленным в техническом задании. При обнаружении несоответствий задокументировать их и устранить.

6. Провести системное тестирование приложения, запустив его на выполнение. Убедиться в соответствии результатов работы компонентов пользовательского интерфейса приложения требованиям, установленным в техническом задании. При обнаружении несоответствий задокументировать их и устранить.

7. Проанализировать код приложения по критериям сложности. В качестве критериев сложности использовать следующие:

- число модулей (классов) приложения;
- суммарное число переменных подпрограмм (методов классов), включая их формальные параметры;
- суммарное количество операторов подпрограмм (методов классов);
- глубина вложенности структурных операторов ветвления и повторения;
- глубина вложенности рекурсий;
- глубина наследования классов.

8. Выполнить описание физических элементов ПС в виде табл. 10. Типы элементов указать согласно табл. 11.

Таблица 10. Перечень узлов программной системы

№	Имя	Тип	Описание
---	-----	-----	----------

Таблица 11. Типы элементов диаграмм развертывания UML

Стереотип	Тип элемента
<<processor>>	Ресурсоемкий узел
<<device>>	Нересурсоемкий узел (устройство)
<<sensor>>	Измерительное устройство (датчик)
<<printer>>	Печатающее устройство (принтер)
<<net>>	Сетевая передающая среда
<<database>>	База данных
<<node>>	Узел, не попадающий под тип, указанный выше в этой таблице

9. Построить диаграмму развертывания UML, выражающую зависимости между узлами ПС и развернутыми на них компонентами специального ПО.

Теоретические положения

5.1. Классификация программного обеспечения ПС

Программное обеспечение (ПО) ПС – это комплекс компьютерных программ, обеспечивающих обработку и передачу данных, а также документация по их конфигурированию и применению. Включает в себя системное, служебное, инструментальное, прикладное и специальное ПО.

Системное ПО – это операционные системы, управляющие функционированием средств вычислительной техники, сетевого оборудования и прикладного ПО.

Служебное ПО – это программные средства, выполняющие функции по обслуживанию компьютерной техники и программ других классов: различные утилиты, средства диагностики, антивирусные программы, и т.п.

Инструментальное ПО – это совокупность программных средств, необходимых для проектирования и разработки ПС: CASE, СУБД, интегрированные среды разработки (IDE), интерпретаторы и трансляторы программ, и т.п.

Прикладное ПО – это совокупность программ, необходимых для обеспечения функционирования программ решения задач управления объектом: внешние библиотеки для прикладных программ, средства защиты данных, офисное ПО, графические пакеты, браузеры, а также иные программы необходимые для полноценного использования специального ПО.

При проектировании ПО проводится анализ и обоснованный выбор инструментального ПО (СУБД, IDE, CASE-средства и др.). Для анализа выбирается не менее 2-х современных систем (языков) программирования или инструментальных средств разработки (IDE), которые пригодны для разработки соответствующих программ. Для каждого средства указываются основные возможности и характерные особенности без подробного описания языка программирования, интегрированной среды и т.п. В результате анализа делается вывод, на основании чего выбирается средство разработки специального ПО.

По результатам проектирования ПО ПС строится его структурная схема в виде древовидной блок-схемы либо организационной диаграммы (рис. 6).

К программному обеспечению, подлежащему разработке в процессе рабочего проектирования, относится категория специального ПО (СПО). СПО – это совокупность программ, непосредственно реализующих алгоритмы решения функциональных задач. СПО разрабатывается при проектировании ПС на базе математического обеспечения и представляет собой программную реализацию моделей, разработанных в при выполнении лабораторной работы №4 и 4.1. Зачастую включает в себя интерфейсные средства: web-страницы, формы, отчеты, модули кода для реализации серверной части СПО, а также запросы, представления, курсоры, хранимые процедуры, макросы и другие модули программного кода. По результатам разработки специального ПО строится его структурная схема в виде диаграммы компонентов UML, либо в произвольной нотации.

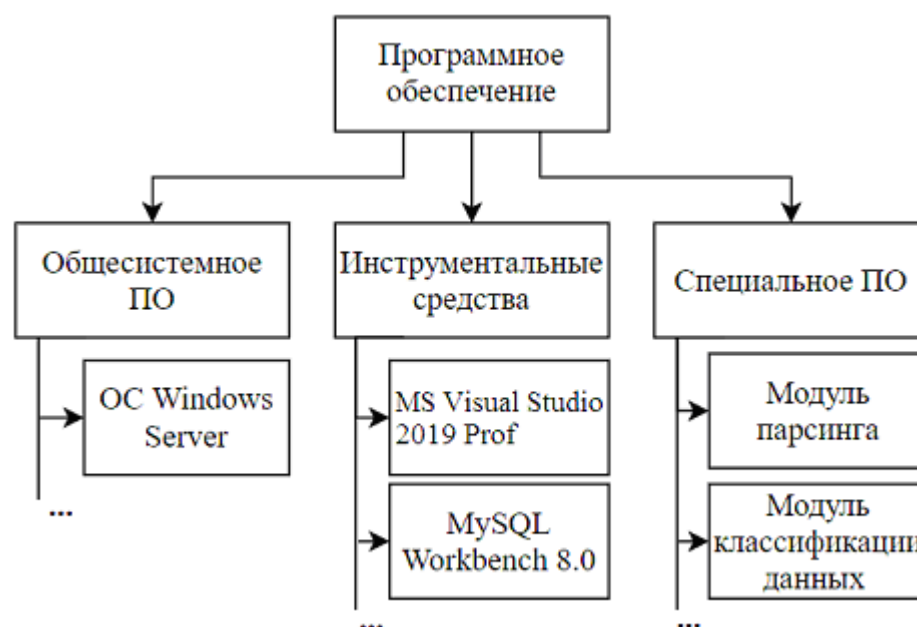


Рисунок 6. Структурная схема программного обеспечения ПС (пример)

5.2. Структура баг-репорта

Баг-репорт (bug report) – это технический документ, описывающий ситуацию или последовательность действий, приведшую к некорректной работе объекта тестирования, с указанием причин и ожидаемого результата. Шаблон упрощенного варианта баг-репорта приведен на рис. 7. Его составляет тестировщик, чтобы разработчикам было понятно, что работает неправильно, насколько дефект критичен и что нужно исправить. Баг-репорты – часть программного процесса.

Шапка	
Короткое описание (Summary)	Короткое описание проблемы, явно указывающее на причину и тип ошибочной ситуации
Проект (Project)	Название тестируемого проекта
Компонент приложения (Component)	Название части или функции тестируемого продукта
Номер версии (Version)	Версия на которой была найдена ошибка
Описание	
Шаги воспроизведения (Steps to Reproduce)	Шаги, по которым можно легко воспроизвести ситуацию, приведшую к ошибке
Фактический Результат (Result)	Результат, полученный после прохождения шагов к воспроизведению
Ожидаемый результат (Expected Result)	Ожидаемый правильный результат

Рисунок 7. Шаблон упрощенного варианта баг-репорта

Баг-репорт – это технический документ. Поэтому язык описания проблемы должен быть техническим. Должна использоваться правильная

терминология при использовании названий элементов пользовательского интерфейса (editbox, listbox, combobox, link, text area, button, menu, popup menu, title bar, system tray и т.д.), действий пользователя (click link, press the button, select menu item и т.д.) и полученных результатах (window is opened, error message is displayed, system crashed и т.д.).

Требования к содержанию отчета

В подразделе 1 проводится анализ и обоснованный выбор общесистемного ПО, необходимого для разработки и эксплуатации ПС. Для анализа выбирается не менее 2-х современных решений, указываются их основные возможности и характерные особенности без подробного описания. В результате анализа делается вывод о выборе. Выбранные компоненты отображаются на схеме по примеру на рис. 6, на которой отдельной ветвью отобразить модули специального ПО.

В подразделе 2 кратко документируется разработанное специальное ПО. Выполняется описание разработанных модулей и компонентов, как по тексту, так и в виде таблиц 8 и 10. Листинг исходных текстов специального ПО может быть представлен в приложении.

В подразделе 3 представляется диаграмма компонентов и выполняется её краткое описание. В описании должны раскрываться существенные детали реализации и взаимодействия компонентов (например, технологии реализации или обмен данными и др.). При этом следует избегать использования стандартного стереотипа «file», а максимально дифференцировать компоненты по типам согласно табл. 9.

В подразделе 4 отражается процедура тестирования специального ПО методом черного ящика. Приводятся листинги модульных тестов и результаты их выполнения. Отражается процедура пользовательского тестирования и приводятся экранные формы пользовательского интерфейса приложения, в том числе отчетов (при наличии). По обнаруженным несоответствиям формируются отчеты (баг-репорты).

Также проводится проверка качества кода компонентов специального ПО. В табличной форме приводятся результаты анализа кода приложения по критериям сложности.

В подразделе 5 представляется диаграмма развертывания и выполняется её краткое описание. В описании должны раскрываться существенные детали развертывания и взаимодействия узлов (например, важные с точки зрения понимания схемы технические характеристики, обмен данными между узлами, отношения узлов с компонентами и др.). При этом следует избегать использования стандартного стереотипа «node», а максимально дифференцировать компоненты по типам согласно табл. 11. Также следует различать на диаграмме узлы и реализованные на их базе компоненты.

Рекомендуемая литература

- [1] – разделы 2.1, 2.2, 2.9, глава 5 (управление качеством).
- [2] – подразделы 1.2.8, 1.3.6, 2.1.12, 2.1.13.
- [3] – разделы 5.4, 6.2, 6.5.
- [4] – подраздел 1.3.2.
- [7] – главы 10, 11.

Контрольные вопросы

1. Что такое программное обеспечение ПС?
2. Каковы отличия между сервисными программами и утилитами?
3. Какие типы программных средств относятся к специальному ПО.
4. Как происходит разработка программного кода ПС?
5. Что такое компиляция и построение программы?
6. Как в интегрированной среде программирования осуществляется обнаружение синтаксических ошибок?
7. Какова связь компонентов ПО с компонентами ИО разрабатываемой ПС?
8. Что такое специальное ПО?
9. Какие бывают типы компонентов специального ПО?
10. Каково назначение диаграммы компонентов UML?
11. Какие бывают типы физических элементов ПС?
12. Каково назначение диаграммы развертывания UML?
13. Определить сложность заданного фрагмента программного кода. Предложить и охарактеризовать критерии анализа кода.
14. Что такое баг-репорт, какова его структура и назначение?

Лабораторная работа 6

Тема: комплексное тестирование ПС

Цель: закрепление методов и способов тестирования ПО и освоение методики комплексного тестирования разработанной программной системы методом белого ящика.

Задание

Выполнить системное пользовательское тестирование работоспособности ПС посредством воздействия на её интерфейсную часть. Для этого необходимо:

1. Выполнить интеграционное тестирование на компонентном уровне разработанных автоматизированных функций ПС, в том числе тестирование взаимодействия компонентов ПС с базой данных, используя подход *Bottom Up Integration*.

2. Согласно функциональным требованиям к ПС, предъявленным при выполнении [лабораторной работы 1](#), разработать несколько сценариев тестирования ПС. Реализовать сценарии в отношении разработанной ПС и задокументировать полученные результаты тестирования. Оценить факторы, оказывающие существенное влияние на эффективность нормального функционирования ПС.

3. Выполнить нагрузочное тестирование ПС и оценить эффективность её автоматизированных функций. В качестве критерия эффективности использовать время выполнения функции. Построить графики зависимости времени выполнения функций от приложенной нагрузки. Оценить факторы, оказывающие существенное влияние на эффективность функционирования ПС под нагрузкой.

4. Выполнить системное тестирование ПС для обеспечения полноты проверки нефункциональных требований, а также для проверки взаимодействия ПС с внешним окружением. Оценить факторы, оказывающие существенное влияние на эффективность взаимодействия.

Теоретические положения

6.1. Виды тестирования ПО

Модульное тестирование – тестируется отдельный модуль, в отрыве от остальной системы. Самый распространенный случай применения – тестирования модуля самим разработчиком, проверка того, что отдельные модули, классы, методы выполняют действительно то, что от них ожидается. Различные среды разработки широко поддерживают средства модульного тестирования. Созданные разработчиком модульные тесты часто включаются в пакет регрессионных тестов и таким образом, могут запускаться многократно.

Интеграционное тестирование – два и более компонентов тестируются на совместимость. Это очень важный вид тестирования, поскольку разные

компоненты могут создаваться разными людьми, в разное время, на разных технологиях. Этот вид тестирования должен применяться самими программистами, чтобы, по крайней мере, удостовериться, что все компоненты программы совместимы в первом приближении. Далее тонкости интеграции могут исследовать тестировщики.

Необходимо отметить, что такого рода "ошибки на стыках" непросто обнаруживать и устранять. Во время разработки все компоненты все вместе не готовы, интеграция откладывается, а в конце обнаруживаются трудные ошибки (в том смысле, что их устранение требует существенной работы: реструктуризации, рефакторинга). Здесь выходом является ранняя интеграция системы и в дальнейшем использование практики постоянной интеграции.

Сценарное тестирование – системный подход к тестированию ПО по предварительно написанным и задокументированным тест-кейсам (сценариям) – последовательностей действий пользователя в приложении.

Системное тестирование – это тестирование всей системы в целом, как правило, через ее пользовательский интерфейс. При этом могут использоваться как ручные, так и автоматические тесты с акцентом на том, как ПО выглядит и работает в целом, удобно ли оно, удовлетворяет ли она ожиданиям заказчика. При этом могут открываться различные дефекты, такие как неудобство в использовании тех или иных функций, забытые или "скудно" понятые требования.

Регрессионное тестирование – тестирование системы в процессе ее разработки и сопровождение на не регресс. То есть проверяется, что изменения системы не ухудшили уже существующей функциональности. Для этого создаются пакеты регрессионных тестов, которые запускаются с определенной периодичностью – например, в пакетном режиме, связанные с процедурой постоянной интеграции.

Нагрузочное тестирование – тестирование системы на корректную работу с большими объемами данных. Например, проверка баз данных на корректную обработку большого (предельного) объема записей, исследование поведения серверного ПО при большом количестве клиентских соединений, эксперименты с предельным трафиком для сетевых и телекоммуникационных систем, одновременное открытие большого числа файлов, проектов и т.д.

Стрессовое тестирование – тестирование системы на устойчивость к непредвиденным ситуациям. Этот вид тестирования нужен далеко не для каждой системы, так как подразумевает высокую планку качества.

Приемочное тестирование – тестирование, выполняемое при приемке системы заказчиком. Более того, различные стандарты часто включают в себя наборы приемочных тестов.

6.2. Сценарное тестирование

Сценарное тестирование предполагает разделение задачи на этапы (подготовка, выполнение, завершение и т.д.) и последовательное выполнение этих этапов.

Особенности этого подхода:

- четкое понимание того, какие функции покрыты тестами;
- уверенность, что все задокументированные тест-кейсы будут пройдены в срок;
- быстрое и легкое подключение нового специалиста на проект благодаря наличию подробных сценариев.

Пример. При тестировании почты, сценарное тестирование может выглядеть так:

- перейти на стартовую страницу;
- нажать на кнопку «Новый»;
- в поле «Кому» написать j.cannegieter@squerist.nl;
- нажать на кнопку «Файл»;
- выбрать документ «Тест1»;
- в поле «Тема» написать «Тестовое вложение <дата>»;
- открыть почтовый ящик j.cannegieter;
- проверить, доставлено ли сообщение и вложение;

Для этого же примера сценарий общего (исследовательского) тестирования может выглядеть так:

- создать и отправить письмо с вложением одному получателю.
- проверить, доставлено ли сообщение и вложение.

6.3. Интеграционное тестирование

Интеграционное тестирование предназначено для проверки связи между компонентами, а также взаимодействия с различными частями системы (операционной системой, оборудованием либо связи между разными системами).

Уровни интеграционного тестирования:

1. Компонентный интеграционный уровень (*Component Integration testing*). Проверяется взаимодействие между компонентами системы после проведения компонентного (модульного) тестирования.
2. Системный интеграционный уровень (*System Integration Testing*), также известное как Тестирование взаимодействия (*Interoperability Testing*). Проверяется взаимодействие системы с другими системами после проведения системного тестирования.

Подходы к интеграционному тестированию:

1. Снизу вверх (*Bottom Up Integration*). Все низкоуровневые модули, процедуры или функции собираются воедино и затем тестируются. После чего собирается следующий уровень модулей для проведения интеграционного тестирования.

2. Сверху вниз (*Top Down Integration*). Вначале тестируются все высокоуровневые модули, и постепенно один за другим добавляются низкоуровневые. Все модули более низкого уровня симулируются заглушками с аналогичной функциональностью, затем по мере готовности они заменяются реальными активными компонентами.

3. Большой взрыв ("*Big Bang*" *Integration*). Все или практически все разработанные модули собираются вместе в виде законченной системы или ее основной части, и затем проводится интеграционное тестирование. Такой подход очень хорош для сохранения времени. Однако если тест кейсы и их результаты записаны не верно, то сам процесс интеграции сильно осложнится, что станет преградой для команды тестирования при достижении основной цели интеграционного тестирования

Например. Установлено, что данные по продажам из портала передаются в аналитическую систему. Тогда результатом интеграционного тестирования является решение о том, что необходимо дополнительно проверить, что:

- а) продажи отправлены в корректном формате;
- б) данные по продажам получены аналитической системой в корректном формате;
- в) потери данных не произошло.

6.4. Системное тестирование

Основной задачей системного тестирования является проверка как функциональных, так и нефункциональных требований в системе в целом. При этом выявляются дефекты, такие как неверное использование ресурсов системы, непредусмотренные комбинации данных пользовательского уровня, несовместимость с окружением, непредусмотренные сценарии использования, отсутствующая или неверная функциональность, неудобство использования и т.д. Для минимизации рисков, связанных с особенностями поведения системы в той или иной среде, во время тестирования рекомендуется использовать окружение максимально приближенное к тому, на которое будет установлен продукт после выдачи. Можно выделить два подхода к системному тестированию:

- на базе требований (*requirements based*). Для каждого требования пишутся тестовые случаи (*test cases*), проверяющие выполнение данного требования.
- на базе случаев использования (*use case based*). На основе представления о способах использования продукта создаются случаи использования системы (*use cases*). По конкретному случаю использования можно определить один или более сценариев. На проверку каждого сценария пишутся тест кейсы, которые должны быть протестированы.

Рекомендуемая литература

- [1] – разделы 1.5, 2.8, глава 5.
- [2] – подразделы 1.3.1, 1.3.2.

[3] – глава 7.

[4] – подраздел 1.3.4.

[5] – главы 11–14.

Контрольные вопросы

1. Перечислите и охарактеризуйте методы обеспечения качества программного обеспечения.
2. Что такое тестирование компьютерной программы?
3. Что такое ожидаемое поведение программы?
4. Какие виды тестов и тестирования программного обеспечения применимы к разработанному программному приложению?
5. Что такое производительность программного обеспечения?
6. Укажите методы и способы оценивания производительности программного обеспечения.
7. В чем заключается идентификация ошибок в программе?
8. Какими способами можно обнаружить логически ошибки в коде?
9. Перечислите показатели, по которым выполняется анализ кода.
10. Что понимается под эффективностью программного обеспечения?
11. Что понимается под сложностью программного обеспечения?
12. Каковы могут быть формы представления результатов оценивания эффективности программного обеспечения?
13. От чего зависит эффективность программного обеспечения?
14. Что такое модульное (компонентное) тестирование?
15. Укажите цели разработки сценариев тестирования.
16. В чем разница между тест кейсом и тестовым сценарием?
17. Перечислите преимущества использования сценариев при тестировании программных продуктов.

Лабораторная работа 7

Тема: документирование и развертывание ПС

Цель: освоение методики документирования ПС

Задание

1. Опираясь на материалы технического задания, разработанного при выполнении [лабораторной работы 1](#), в частности, на нефункциональные требования к ПС, выполнить описание разработанной ПС по следующей схеме:

- назначение ПС;
- технические характеристики ПС;
- принцип работы ПС;
- качество поставки ПС;
- меры безопасности при эксплуатации ПС.

2. Составить руководство пользователя ПС для всех категорий пользователей, которые показаны на диаграмме требований UML, построенной при выполнении [лабораторной работы 1](#). Оценить факторы, оказывающие существенное влияние на качество эксплуатации ПС.

3. Спланировать развертывание ПС и определить состав квалифицированной группы внедрения и сопровождения ПС. План внедрения должен быть предельно конкретизирован относительно объекта и процесса компьютеризации. Оценить факторы, оказывающие существенное влияние на качество развертывания ПС.

4. Подготовить предложения по возможному совершенствованию, развитию и модернизации ПС.

Теоретические положения

7.1. Документирование ПО

Документирование относится к категории вспомогательных процессов жизненного цикла информационной системы, обеспечивающих выполнение основных процессов жизненного цикла ПС.

При осуществлении рабочего проектирования создаются следующие виды документации:

- функциональные спецификации – описывают функционирование отдельных компонентов ПС и системы в целом в отношении реализации функциональных требований к системе;
- проектные спецификации – раскрывают проектные решения, которые необходимы заказчику для выполнения функций сопровождения ПС;
- руководство пользователя – документ, регламентирующий действия пользователя при работе с ПС.

Документирование часто сопровождается диаграммами UML, которые позволяют визуально описывать приложение, наглядно представлять архитектуру и требования к приложению.

Основное требование к документации – она должна быть понятна и соответствовать функциональной и обеспечивающей структуре ПС. Основные ошибки при документировании ПС: нечеткие формулировки, игнорирование аудитории, для которой предназначена данная документация, а также пропуск важных аспектов, связанных с нефункциональными требованиями.

7.2. Руководство пользователя

Руководство пользователя – это своего рода инструкция по эксплуатации приложения конечными пользователями. Предназначено для всех людей, принимающих участие в настройке и эксплуатации программного приложения.

Примерная структура руководства пользователя:

1. «Назначение приложения». Здесь задаются общие понятия, необходимые навыки для работы с системой:

- название приложения и номер версии;
- класс ПС, для которых предназначено приложение;
- степень универсальности приложения для решения поставленных задач;
- характеристики пользовательского интерфейса;
- уровень подготовки пользователей для эффективного взаимодействия с приложением;

2. «Установка приложения». В этой части описывается процесс установки и базовой настройки системы.

3. «Концепция приложения». Приводится общая структура системы, приводится справочник терминов и понятий, которые используются в интерфейсах управления системой и данном руководстве, описывается процесс функционирования в общем виде.

4. «Функциональный состав приложения». Рассматривается процесс решения задач на объекте в рамках функционального состава ПС. По ходу описания приводятся экранные формы всех интерфейсных окон / страниц, посредством которых пользователи взаимодействуют с приложением и устанавливаются ссылки на них.

5. «Инструменты и настройки приложения». Описывается назначение и применение настроек приложения, работа с модулями, обновлениями и полезными инструментами системы.

6. «Получение помощи». Приводятся способы решения возникших проблем и получения помощи (Help).

Рекомендуемая литература

- [1] – разделы 3.6.
- [2] – раздел 1.4.
- [4] – подраздел 1.3.5.
- [5] – главы 15, 17.

Контрольные вопросы

1. Что такое документирование ПС и каково его назначение?
2. Какие элементы включает в себя структура документации к ПС?
3. Что такое руководство пользователя?
4. Перечислите основные структурные элементы руководства пользователя.
5. Каково назначение руководства пользователя?
6. Какие задачи решаются на этапе эксплуатации ПС?
7. В чем заключается планирование развертывания ПС?
8. Перечислите основные этапы развертывания ПС.
9. Каковы основные проблемы развертывания ПС?
10. Перечислите основные факторы успеха при развертывании ПС.

Лабораторная работа 8

Тема: управление рисками ПС.

Цель: освоение методики идентификации, анализа и картографирования рисков факторов на стадиях жизненного цикла ПС, выработка необходимых превентивных мероприятий для нормализации рисков факторов ПС.

Задание

1. Идентифицировать риски проекта в категориях разработки, внедрения и применения ПС. В качестве рисков разработки использовать факторы эффективности функционирования ПС, выявленные при выполнении [лабораторной работы 6](#). Риски разработки также следует искать среди задач со значительными временными характеристиками (трудоемкость, время оценивания, сроки). В качестве рисков внедрения и применения использовать факторы соответственно развертывания и эксплуатации ПС, выявленные при выполнении [лабораторной работы 7](#).

2. Провести оценивание рисков, используя следующие две ранговые шкалы: 1) вероятности реализации риска и 2) существенности последствий риска.

3. Заполнить карту рисков, используя полученные результаты их идентификации и оценивания. Провести ранжирование рисков.

4. На основе полученных данных составить перечень мероприятий для нормализации рисков факторов ПС.

Теоретические положения

8.1 Методика риск-менеджмента программного процесса

Любая организация, являясь элементом нестабильной социально-экономической среды общества, испытывает необходимость перманентного управления рисками. Важным фактором эффективности здесь является рисков фактор, определяющий потенциал успешности функционирования объекта компьютеризации. Исследование рисков фактора позволяет идентифицировать основные риски, определить их возможные последствия, предложить превентивные подстроечные мероприятия по купированию возможных негативных последствий рисков.

Понимая риск как степень неопределённости функционирования объекта компьютеризации в ситуации, когда имеется возможность отклонения результатов от предполагаемой цели, следует также тщательно проанализировать область применения ИС. При этом необходимо выявить риски, связанные с низкой эффективностью принятия решений и/или высокой субъективностью результатов реализации принятых управленческих решений.

Требования к содержанию отчета

8.2. Идентификация рисков

В подразделе 1 следует разделить риски на 3 категории, как показано в табл. 12. Необходимо выявить 3–5 рисков каждой из трех категорий, актуальных для проектируемой ИС. Это должны быть риски, природа которых наиболее очевидна. Примеры некоторых рисков показаны в табл. 12. Риски применения (категория «П») чаще носят сугубо частный характер сообразно области применения. Например, риск нарушения хода учебных занятий в соответствии с индивидуальными образовательными траекториями обучающихся.

Таблица 12. Классификация рисков для решения поставленной задачи, категории рисков и примеры рисков по категориям

Класс риска	Подкласс рисков	Примеры рисков с указанием шифра
Риски разработки ИС для обеспечения успешности функционирования объекта (категория «Р»)	–	Р-1 – увеличение нагрузки на работников объекта на этапе его обследования Р-2 – некорректные требования к ПО, сформулированные заказчиком Р-3 – недостаточное финансирование проекта
Риски внедрения ИС и её применения для управления объектом (выработки и применения управленческих решений по приведению объекта в заданное состояние)	Риски нарушения функционала реализации разработанных решений – системные риски (категория «С»)	С-1 – загрузка недостоверных данных для анализа, либо их отсутствие С-2 – нестабильная работа сети Интернет С-3 – недостаточность вычислительной мощности
	Риски применения на объекте выработанных в нормальных условиях управленческих решений (категория «П»)	П-1 – слабая синхронизация бизнес-процессов и разработанных решений П-2 – недостаточная квалификация пользователей П-3 – отказ от реализации управленческих решений

Особое внимание при анализе рисков следует уделить так называемым системным рискам (категория «С» в табл. 12) как возможным рискам нарушения функционала либо появления системных дисфункций ИС. Как правило, такие риски наиболее критичны, так как приводят к частичной либо полной потере эффективности и результативности ИС. Описание рисков и их последствий свести в таблицу 13.

Таблица 13. Риски и последствия рисков

Шифр	Риск	Последствия риска

8.3. Качественное оценивание рисков

В подразделе 2 необходимо кратко описать процедуру оценивания рисков. Для оценивания по обеим шкалам используется трехбалльная ранговая шкала, отраженная на рис. 8. Результаты оценивания рисков представить в таблицах 14 и 15.



Рисунок 8. Частный пример карты рисков

В табл. 14 диапазон вероятности реализации определить априорно.

Таблица 14. Шкала оценивания вероятности реализации риска

Шифр	Вероятность реализации	Диапазон вероятности реализации

В табл. 15 существенность последствий кратко описать в пределах 10-20 слов для каждого риска.

Таблица 15. Шкала оценивания существенности последствий риска

Шифр	Оценка существенности	Существенность последствий

В подразделе 3 необходимо отразить процедуру построения карты рисков. Для удобства составления карты рисков необходимо вынести в табл. 16 выявленные риски в соотнесении по шкалам оценивания существенности последствий и вероятности наступления риска. Ранжирование проводить сначала по вероятности, а затем по существенности рисков.

Таблица 16. Соотнесение риска с вероятностью реализации и существенностью последствий

Шифр	Вероятность реализации	Оценка существенности

Частный пример карты рисков представлен на рис. 8. На карте рисков вероятность (или частота) отображается по горизонтальной оси, а существенность последствий по вертикальной оси. В этом случае вероятность появления риска увеличивается слева направо по горизонтальной оси, а существенность последствий риска увеличивается сверху вниз по направлению вертикальной оси.

Красным цветом на карте выделены области критических рисков: риски, у которых высокая вероятность появления и тяжелая существенность последствий свершения риска. На эти риски управленцу необходимо обратить внимание в первую очередь: либо смягчить последствия, либо вообще исключить свершение этих рисков.

Черная жирная линия над областью допустимых рисков – это граница толерантности или критическая граница терпимости к риску. Риски, лежащие на границе толерантности, необходимо смещать в область управляемых рисков, чтобы они не перешли в область критических рисков.

Оранжевым цветом выделены риски границы терпимости, у которых средняя частота появления и терпимые последствия свершения рискового события. Эти риски решаются управленцем во вторую очередь, так как у них терпимые последствия и по сравнению с критическими рисками они могут подождать. Однако не следует забывать, что эти риски могут легко перейти в красную область критических рисков.

Зеленым цветом выделены риски, которые редко свершаются и имеют незначительные последствия. С этими рисками работают в последнюю очередь, так как они не несут больших потерь и являются управляемыми в рабочем порядке.

8.4. Воздействие на риски

В подразделе 4 на основании полученного материала исследования рискового фона необходимо сделать заключение о подготовке и реализации необходимых превентивных мероприятий, достаточных для приведения объекта компьютеризации к состоянию умеренного рискового фона, находящегося на карте рисков левее линии толерантности. Заключение должно быть развернутым и содержать:

- вывод о целесообразности применения ИС по назначению;
- прогноз изменения карты рисков в перспективе;
- решения по возможным мероприятиям, оптимизирующим рисковый фон на объекте управления.

Выделенные мероприятия целесообразно представить в виде табл. 17. Последствия риска могут быть продублированы из табл. 13 или быть

модифицированы с учетом проведенного исследования рискового фона при выполнении пунктов задания 2 и 3.

Таблица 17. Последствия рисков и мероприятия для оптимизации рискового фона объекта управления

Шифр риска	Последствия риска	Мероприятия по купированию последствий риска

Следует отметить, что реализация качественного и высоко эффективного риск-менеджмента – задача сложная и требует применения нетривиального математического аппарата и построения математической модели мониторинга и регулирования рискового фона объекта управления.

Рекомендуемая литература

- [1] – глава 7.
- [4] – раздел 2.2.
- [5] – глава 10.

Контрольные вопросы

1. Что такое проектный риск?
2. Что такое карта рисков?
3. Какие методы ранжирования существуют?
4. Что такое шкала существенности карты рисков?
5. Что такое шкала оценивания вероятности возникновения риска?
6. На какие риски стоит обращать внимание в первую очередь?
7. Что такое критическая граница терпимости риска?

Список рекомендуемой литературы

Основная литература:

1. Полетайкин, А.Н. Методология и технология разработки программных систем: методы и модели программной инженерии: учебное пособие / А.Н. Полетайкин, Н.Ю. Добровольская; Министерство науки и высшего образования Российской Федерации, Кубанский государственный университет. – Краснодар: Кубанский гос. ун-т, 2025. – 229 с. Режим доступа: <http://212.192.134.46/MegaPro/Download/MObject/1389>.
2. Полетайкин, А.Н. Проектирование программных систем: учебное пособие / А.Н. Полетайкин, Н.Ю. Добровольская; Министерство науки и высшего образования Российской Федерации, Кубанский государственный университет. – Краснодар: Кубанский гос. ун-т, 2025. – 190 с. Режим доступа: <http://megapro.kubsu.ru/MegaPro/Download/MObject/1444>.
3. Полетайкин А. Н. Социальные и экономические информационные системы: законы функционирования и принципы построения : Учеб. пособие / Сиб. гос. ун-т телекоммуникаций и информатики. - Новосибирск : СибГУТИ, 2016. - 241 с. Режим доступа: https://sibsutis.ru/upload/13e/up_is.pdf.
4. Ехлаков, Ю. П. Введение в программную инженерию: Учебное пособие [Электронный ресурс] / Ю. П. Ехлаков. — Томск: ТУСУР, 2011. — 148 с. — Режим доступа: <https://edu.tusur.ru/publications/141>.
5. Липаев В.В. Программная инженерия. Методологические основы: учебник. – М.: «ТЕИС», 2006 – 608 с.
6. Батоврин В.К. Системная и программная инженерия. Словарь-справочник: учеб. пособие для вузов. - М.: ДМК Пресс, 2010. - 280 с.
7. Леоненков А. В. Самоучитель UML. - 2-е изд., перераб. и доп. — СПб.: БХВ-Петербург, 2004. - 432 с.
8. Мартин Р. Чистый код. Создание, анализ и рефакторинг : монография. – СПб.: ПИТЕР, 2012. – 464 с.
9. Орлов С.А. Технологии разработки программного обеспечения : Учебник для вузов. – СПб. : ПИТЕР, 2002. – 463 с.

Дополнительная литература:

10. Иванова Г.С. Технология программирования : учебник – 3-е изд., перераб. и доп. – М. : Изд-во МГТУ им. Н. Э. Баумана, 2006. – 335 с.
11. Денис М. Ахен, Арон Клауз, Ричард Тернер. СММІ: Комплексный подход к совершенствованию процессов. Практическое введение в модель / Пер. с англ. – М.: «МФК», 2005. – 330 с.
12. Гагарина Л.Г., Кокорева Е.В., Виснадул Б.Д. Программная инженерия : учеб. пособие; под ред. Л.Г. Гагариной. – М.: ИД "ФОРУМ" : ИНФРА-М, 2012. – 399 с.
13. Никитин В.А. Управление качеством на базе стандартов ИСО 9000:2000 : монография. – СПб. : ПИТЕР, 2002. – 262 с.

14. Босуэлл Д., Фаучер Т. Читаемый код или Программирование как искусство : монография. – СПб. : ПИТЕР, 2012. – 203 с.
15. Лавров С.С. Программирование. Математические основы, средства, теория : монография. – СПб. : БХВ-Петербург, 2001. – 317 с.
16. Вернер М. Основы кодирования : учебник / пер. с нем. Д.К. Зигангирова. – М. : Техносфера, 2004. – 286 с.
17. Макконелл Дж. Основы современных алгоритмов : учеб. пособия / пер. с англ. под ред. С.К. Ландо; доп. М.В. Ульянова. – М. : Техносфера, 2004. – 366 с.
18. Губарев А.В. Информационное обеспечение системы менеджмента качества : моногр. – Москва : Горячая линия-Телеком, 2013. – 132 с. : ил.
19. Лодон Дж. Управление информационными системами : учебник. – 7-е изд. – СПб. : ПИТЕР, 2005. – 910 с.
20. Кронрод А.С. Беседы о программировании : научно-популярная литература / предисл. Л.А. Кронрод; послеслов. В.Л. Арлазарова. – 2-е изд., стереотип. – М. : УРСС, 2004. – 246 с.
21. Кузнецов С.М., Малозёмов Б.В. Программирование и основы алгоритмизации : учеб. пособие. – Новосиб. гос. техн. ун-т. – Новосибирск : Изд-во НГТУ, 2006. – 199 с. : ил.
22. Кнут Д.Э. Искусство программирования: В 3 т. : Пер. с англ. Т.1:Основные алгоритмы : монография / Под общ. ред. Ю.В.Козаченко. – М. : Издат.дом "Вильямс", 2002. – 712 с.

ГОСТы:

23. ГОСТ Р ИСО/МЭК 12207–99. ГОСУДАРСТВЕННЫЙ СТАНДАРТ РОССИЙСКОЙ ФЕДЕРАЦИИ. Информационная технология. ПРОЦЕССЫ ЖИЗНЕННОГО ЦИКЛА ПРОГРАММНЫХ СРЕДСТВ.
24. ГОСТ Р ИСО/МЭК 15504-1-2009. Информационные технологии. Оценка процессов. Часть 1. Концепция и словарь Текст. Введ. 14.09.2009. – М. : Стандартинформ, 2010. – 19 с.
25. ГОСТ 34.601–90. Автоматизированные системы. Стадии создания. В кн.: Информационная технология. Комплекс стандартов и руководящих документов на автоматизированные системы. М.: Комитет стандартизации и метрологии СССР,1991. – с.45-52.
26. ГОСТ 34.602–89. Техническое задание на создание автоматизированной системы. В кн.: Информационная технология. Комплекс стандартов и руководящих документов на АС. М.: Комитет стандартизации и метрологии СССР,1991. – с.15-29
27. РД 50–34.698–90. Автоматизированные системы. Требования к содержанию документов. В кн.: Информационная технология. Комплекс стандартов и руководящих документов на АС. М.: Комитет стандартизации и метрологии СССР,1991. – с.66-104.
28. ГОСТ 19.701–90 (ИСО 5807-85). Схемы алгоритмов, программ, данных и систем. Госстандарт СССР, 1990. – 20 с.

Приложение А – Варианты индивидуальных заданий

1. Обменный пункт: сотрудники пункта, виды валют, курсы валют, операции обмена.
2. Ювелирный магазин: названия изделий, комитенты (кто сдал изделия на комиссию), журнал сдачи изделий на продажу, журнал покупки изделий.
3. Поликлиника: врачи, пациенты, виды болезней, журнал учета прихода пациентов.
4. Кондитерский магазин: виды конфет, поставщики, торговые точки, журнал поступления и отпуска товара.
5. Автобаза: автомашины, водители, рейсы, журнал выезда машин на рейсы.
6. Парикмахерская: клиенты, прайс услуг, сотрудники, кассовый журнал.
7. Школа: учителя, предметы, ученики, журнал успеваемости.
8. Оплата услуг на дачных участках: виды услуг, список владельцев, сотрудники управления, журнал регистрации оплат.
9. Гостиница: проживающие, сотрудники гостиницы, номера, журнал регистрации проживающих.
10. Книжный магазин: авторы, книги, продавцы, покупатели, регистрация продаж.
11. Ремонтная мастерская: виды работ, исполнители, заказы на ремонт, заказчики.
12. Аптечный киоск: номенклатура лекарств, работники аптеки, покупатели, журнал регистрации продаж.
13. Выставка: стенды, стендисты, экскурсии, посетители.
14. Охранная служба: список постов охраны, список охранников, журнал выхода на дежурство, журнал учета замечаний.
15. Столовая: продукты, блюда, меню, журнал заказов.
16. Фотомастерская: заказчики работ, прайс работ, журнал поступления заказов, исполнители.
17. Ветеринарная лечебница: список животных, список болезней, список хозяев, журнал посещений.
18. Сельское хозяйство: список растений, список угодий, список работников, журнал посевной.
19. Холдинг: список регионов, список предприятий, список показателей, журнал учета отчетных данных.
20. Фонды предприятия: список основных средств, список категорий основных средств, список материально ответственных лиц, журнал учета состояния основных средств.
21. Учет расхода материалов в компании: список статей затрат, список сотрудников, журнал учета расхода канцтоваров, список департаментов.
22. Фильмотека: список фильмов, список клиентов, список библиотекарей, журнал выдачи фильмов.
23. Цирк: список категорий артистов, список артистов, журнал выхода артистов на работу, список цирковых площадок.

24. Спортивные заведения: список спортсменов, список видов спорта, список стадионов, журнал учета выступлений спортсменов.
25. Компьютерные занятия: список слушателей курсов, список предметов, список преподавателей, журнал учета успеваемости.
26. Сбор урожая: список видов продукции, список сборщиков, список бригад, журнал учета сбора урожая.
27. Фирма по обслуживанию населения: список заказчиков, список товаров, список разносчиков, журнал заказов.
28. Партийная работа: список членов партии, список мероприятий, журнал учета выхода на мероприятие, список городов
29. Экономическая база данных: список регионов, список показателей, список отраслей, отчетные статистические данные.
30. Журнальные статьи: список тем, список авторов, список названия статей, список журналов.
31. Анализ причин заболеваемости: список больных, список болезней, список районов, журнал учета заболевших.
32. Отдел кадров: список сотрудников, штатное расписание, список отделов, журнал перемещения сотрудников по службе.
33. Расчет нагрузки на преподавателя: список преподавателей, список кафедр, предметов, журнал нагрузки.
34. Проектные работы: список проектов, список специалистов, список должностей, журнал учета работ.
35. Учет компьютерного оборудования: список типов оборудования, список материально ответственных лиц, список департаментов, журнал регистрации выдачи оборудования.
36. Прививки детям: список прививок, список детей, список родителей, журнал учета сделанных прививок.
37. Начисление налогов в бюджет: виды налогов, список отраслей, список предприятий, журнал учета поступления налогов.
38. Экспертная система: список оцениваемых объектов, список экспертов, список регионов, журнал учета оценок.
39. Ремонтная мастерская электронного оборудования: список работ, список мастеров, список запасных частей, журнал учета выполненных работ, список поступившего оборудования.
40. Магазин по продаже автомобилей: список производителей, список автомобилей, журнал поступления автомобилей, список водителей пригнавших машину.
41. Автомобильный гараж: список владельцев, список автомобилей, список сторожей, журнал прихода и ухода автомобилей.
42. Учет криминогенной ситуации в городе: список районов, список типов преступлений, список дежурных, журнал регистрации преступлений.
43. Система здравоохранения: список регионов, список санаториев, список пенсионеров, журнал регистрации выдачи путевок в санатории.
44. Туристические агентства: список туров, список стран, список клиентов, журнал регистрации продаж туров.

45. Продажа билетов на рейсы: список рейсов, прайс билетов, список компаний, журнал продаж билетов.
46. Продажа пиломатериалов: виды пиломатериалов, регионы поставщики, список заказчиков, журнал учета продаж пиломатериалов.
47. Склад металлоконструкций: прайс товара металлоконструкций, список поставщиков, список продавцов, журнал учета продаж.
48. Система поддержки решений: список экспертов, список тем обсуждений, список департаментов, журнал учета предложений.
49. Детский сад: список родителей, список детей, список групп, журнал посещения детского сада.
50. Дом творчества молодежи: список кружков, список руководителей, список детей, журнал регистрации посещения кружков.

Приложение Б – Примерное описание объекта компьютеризации

Характеристика объекта компьютеризации

Объектом компьютеризации является ПАО Банк «ФК Открытие». Банк «Открытие» – крупнейший частный банк в России и четвертый по размеру активов среди всех российских банковских групп. Работает на финансовом рынке с 1993 года [1].

Активы банка и его дочерних компаний по международным стандартам финансовой отчетности (МСФО) на 30 июня 2016 года составили 3 087,8 млрд рублей, собственный капитал – 217,8 млрд рублей.

«Открытие» – универсальный коммерческий банк с устойчивой диверсифицированной структурой бизнеса и качественным управлением капиталом.

Банк развивает следующие ключевые направления бизнеса:

- корпоративный;
- инвестиционный;
- розничный;
- малый и средний бизнес;
- приват банкинг.

Особое внимание «Открытие» уделяет высокотехнологичным сервисам. Так, в рамках проекта «Рокетбанк» предлагается полностью дистанционный сервис для физических лиц, а в рамках проекта «Точка» – полностью дистанционное обслуживание для предпринимателей.

Банк «Открытие» был сформирован в результате интеграции более чем десяти банков различного масштаба, в том числе таких крупных федеральных, как НОМОС-БАНК, Ханты-Мансийский банк и банк «Петрокоммерц».

Банк «Открытие» входит в список десяти системообразующих кредитных организаций, утвержденный Центральным банком, а также в список крупнейших компаний мира FORBES Global 2000.

Надежность банка подтверждена рейтингами международных агентств S&P Global (BB-) и Moody's Investors Service (Ba3).

Клиентская база объединенного банка насчитывает свыше 30 000 корпоративных клиентов, 165 000 клиентов малого бизнеса и около 3,2 млн физических лиц, в том числе премиальных клиентов. 470 отделений банка различного формата расположены в 53 экономически значимых регионах России. Значительная часть бизнеса сосредоточена в Москве, Санкт-Петербурге, Тюменской области, Екатеринбурге, Новосибирской области, Хабаровском крае, Волгоградской области.

Ключевым акционером банка «Открытие» является «Открытие Холдинг», который владеет 64,7% голосующих акций (см. рис.). Бенефициарами «Открытие Холдинг» являются: Вадим Беляев, Группа «ИФД Капиталь», Банк

«ВТБ», Группа «ИСТ», Рубен Аганбегян, Александр Мамут. Бумаги банка также находятся в свободном обращении на Московской бирже.



Рисунок – Структура ОАО «Открытие Холдинг»

Главой Наблюдательного совета банка «Открытие» является Дмитрий Ромаев, председателем Правления – Евгений Данкевич.

Банк «Открытие» активно участвует в реализации социально значимых проектов, поддерживая такие направления, как наука (Политехнический музей), образование (Московский государственный университет им. Ломоносова, Европейский университет в Санкт-Петербурге), культура (Малый театр, сотрудничество с автором мультфильма «Ёжик в тумане» Юрием Норштейном), здравоохранение (Фонд помощи хосписам «Вера»), спорт (стратегическое партнерство с ФК «Спартак-Москва», Югорский лыжный марафон).

Для компьютеризации было выбрано отделение банка в г. Новосибирске, располагающееся по адресу: 630102, г. Новосибирск, ул. Кирова, д. 44.

Контактный телефон: (383) 325-10-85.

Управляющий филиалом: Ермилов Валерий Николаевич.

Главный бухгалтер: Жданов Алексей Владимирович.

Филиал обслуживает как частные, так и юридические лица.

Услуги частным лицам включают в себя:

- срочные переводы по системе «Western Union» в рублях (по России) и в валюте в любую точку мира;
- прием коммунальных платежей;
- оплата услуг операторов мобильной связи;
- оплата кредитов, полученных в других банках;
- автокредитование;
- кредиты по пластиковым картам;
- открытие и ведение текущих рублёвых и валютных счетов;

- предоставление индивидуальных банковских сейфов;
- оформление и обслуживание пластиковых карт;
- денежные вклады;
- ипотечное кредитование;
- инвестиции;
- страхование;
- пенсионный калькулятор;
- сейфовые ячейки;
- операции с иностранной валютой и чеками.

Услуги корпоративным клиентам включают в себя:

- расчётно-кассовое обслуживание, в том числе по системе «Банк-Клиент»;
- широкий выбор кредитных продуктов (кредитные линии, овердрафт, вексельные кредиты, лизинг);
- оперативное проведение валютно-финансовых операций;
- вклады и депозиты для корпоративных клиентов;
- организация процедуры выплаты зарплаты с помощью пластиковых карт;
- оформление пластиковых корпоративных карт международных платёжных систем;
- консалтинговые услуги;
- дистанционное обслуживание;
- инвестиции.

Процессами компьютеризации являются: расчёт ежемесячного платежа для физических лиц и оценка их кредитоспособности при ипотечном кредитовании.

Ипотечное кредитование – долгосрочный кредит, предоставляемый юридическому или физическому лицу банками под залог недвижимости (земли, производственных и жилых зданий, помещений, сооружений). Самый распространенный вариант использования ипотеки в России – это покупка физическим лицом квартиры в кредит. Закладывается при этом, как правило, вновь покупаемое жилье, хотя можно заложить и уже имеющуюся в собственности квартиру [2].

Цель информатизируемых процессов: рассчитать ежемесячный платёж и проверить кредитоспособность заёмщика.

Ограничения на операции: возраст клиента должен быть не менее 18 лет и не более 65 лет на момент выплаты последнего платежа.

Перечень ссылок

- 1 Банк «Финансовая корпорация Открытие» [Офиц. сайт]. URL: <https://www.open.ru> (дата обращения: 17.05.2017).
- 2 IPOhelp [Офиц. сайт]. URL: <http://www.ipohelp.ru/manual.html> (дата обращения: 10.02.2017).

Приложение В – Образец титульного листа
(масштабируется к формату А4)

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Кафедра информационных технологий

ОТЧЕТ

о выполнении лабораторной работы №__
по дисциплине «Программная инженерия»

Выполнил: ст. гр. ____

<Фамилия И.О.>

Проверил: <должность>

<Фамилия И.О.>

Краснодар

20__